COMP3011
计算机体系结构
2.1 存储器层次结构设计

授课教师：王强
邮箱：qiang.wang@hit.edu.cn



第2.1节 存储器层次结构设计

1、Cache的基本概念

2、Cache的基本优化方法

目录

➢ Cache的基本概念

- 存储系统的层次结构
- Cache基本知识

➢ Cache的基本优化方法

- Cache性能分析
- 降低失效率
- 减少失效开销
- 缩短命中时间

目录



➢ Cache的基本概念

- 存储系统的层次结构
- Cache基本知识

➢ Cache的基本优化方法

- Cache性能分析
- 降低失效率
- 减少失效开销
- 缩短命中时间

如何获取粮食？



家里的米桶



超市的大米货物



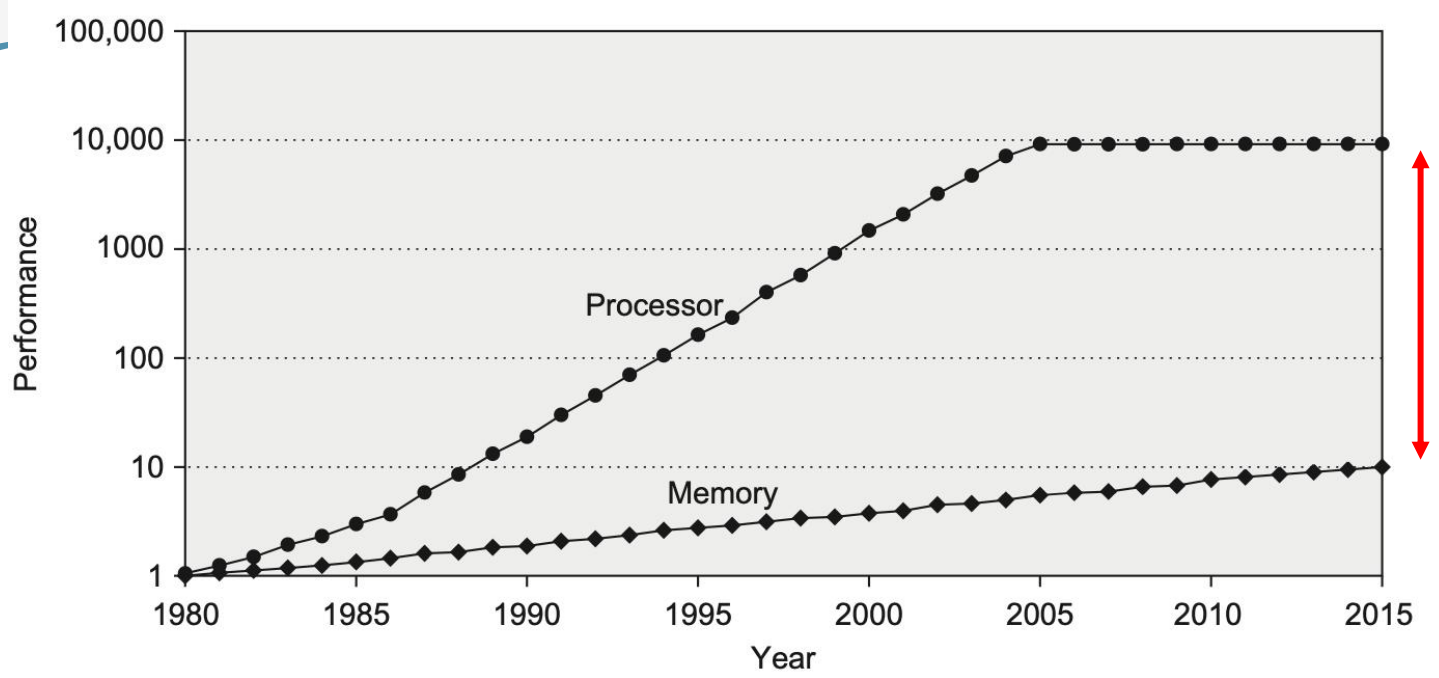
国家的粮仓

获取粮食的时间成本？不同途径粮食的数量？

1、Cache的基本概念 – 存储系统分层结构

- **存储系统设计是计算机体系结构设计的关键问题之一**
  - 价格，容量、速度的权衡
- **用户对存储器的“容量，价格和速度”要求是相互矛盾的**
  - 速度越快，每位价格就高
  - 容量越大，每位价格就低
  - 容量越大，速度就越慢
  - 目前主存一般由DRAM构成
- **处理器与DRAM之间的性能差异越来越大**
  - CPU性能提高大约60%/year
  - DRAM 性能提高大约 9%/year

处理器与DRAM之间的性能差异

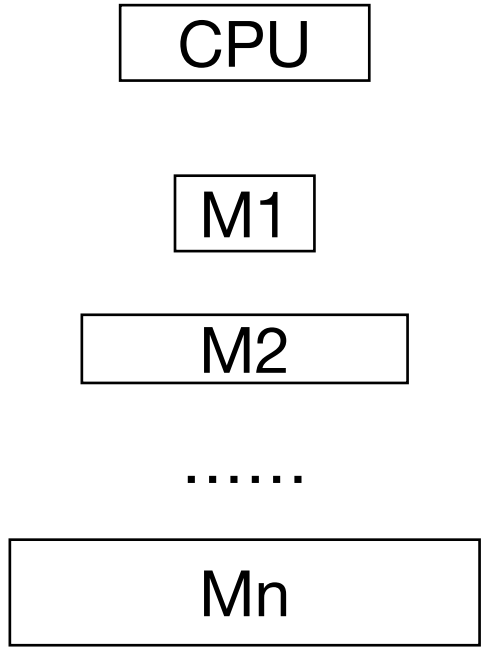


处理器与内存的性能差异

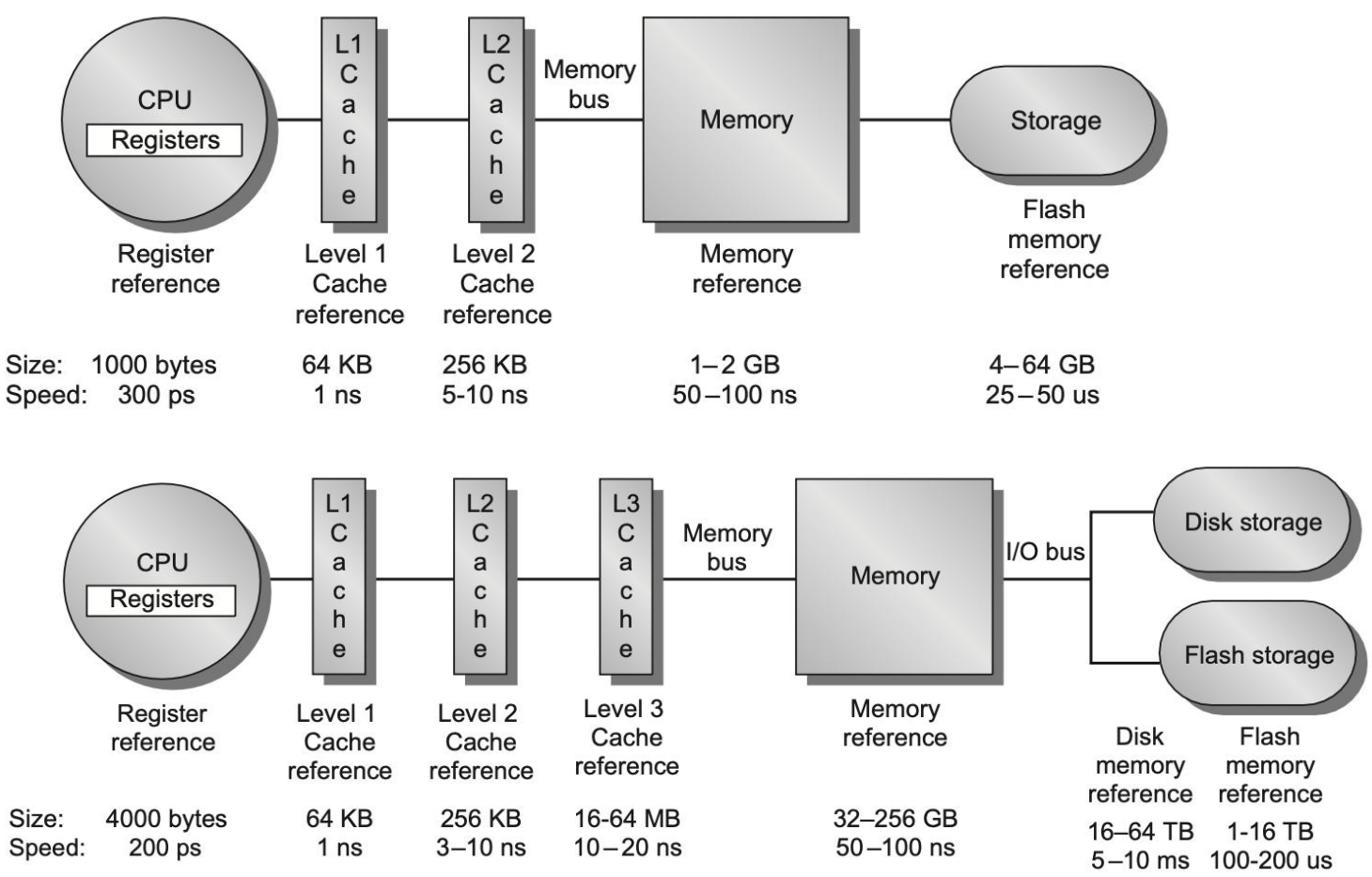
- Time of a full cache miss in instructions executed
  - 1st Alpha : 340 ns/5.0 ns = 68 clks x 2 or 136 instructions
  - 2nd Alpha :266 ns/3.3 ns = 80 clks x 4 or 320 instructions
  - 3rd Alpha :180 ns/1.7 ns =108 clks x 6 or 648 instructions

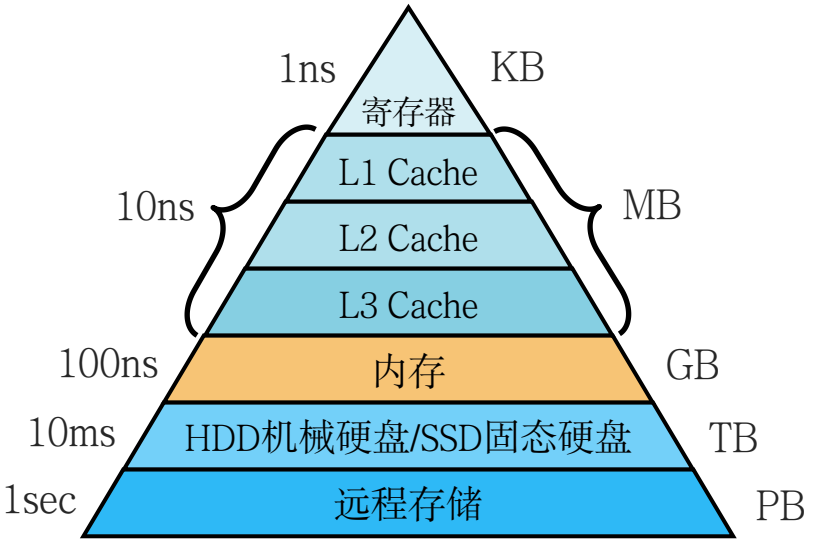
存储系统的设计目标

- **目标：针对典型应用平均访存时间最短**
- **途径：通过优化存储系统的组织**
- **解决方法：多级层次结构**
  - M1速度最快，容量最小，每位价格最高
  - Mn速度最慢，容量最大，每位价格最低
- **存储系统接近M1的速度，容量和价格接近Mn**

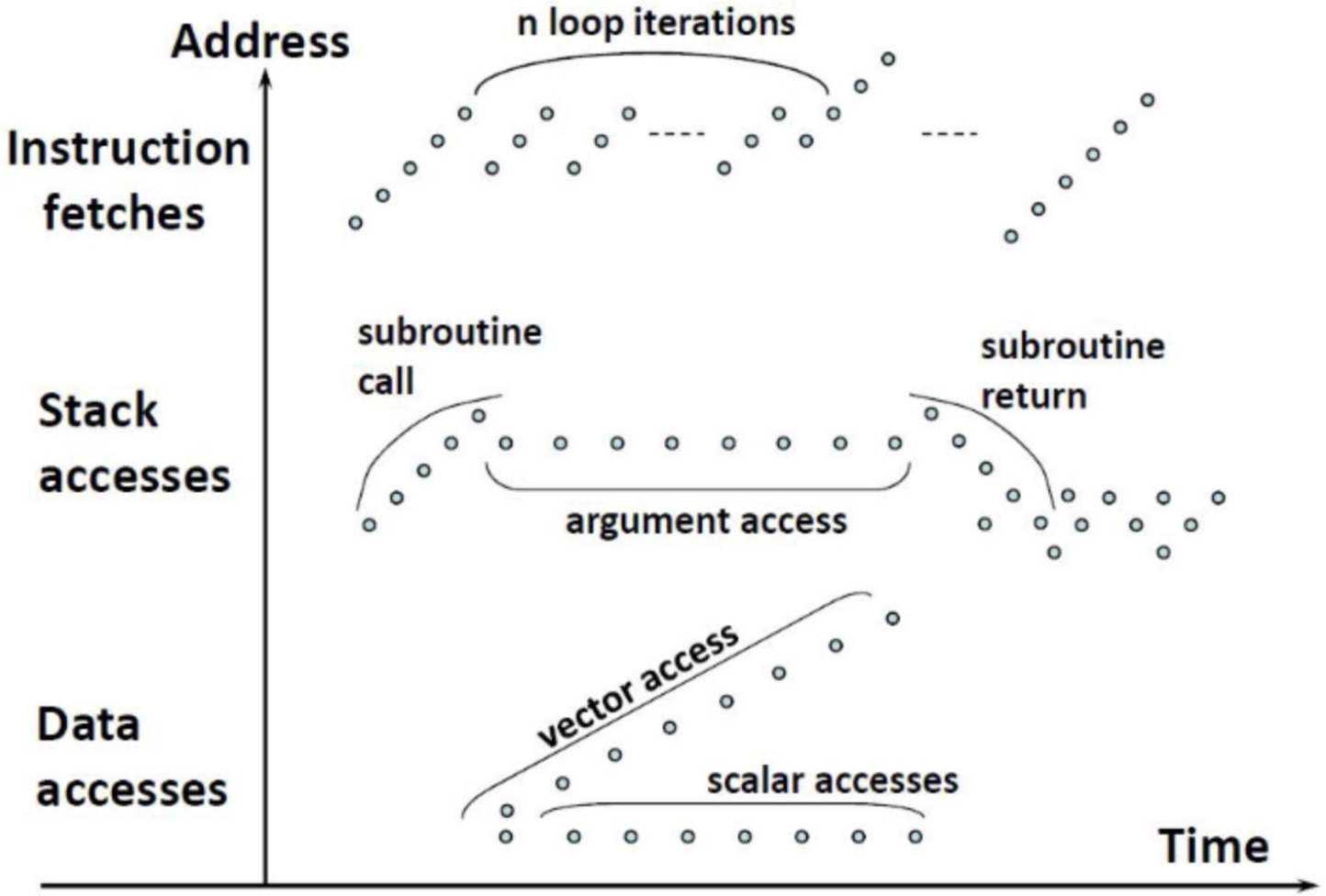


计算机存储系统的多级存储层次





典型的存储器访问模式

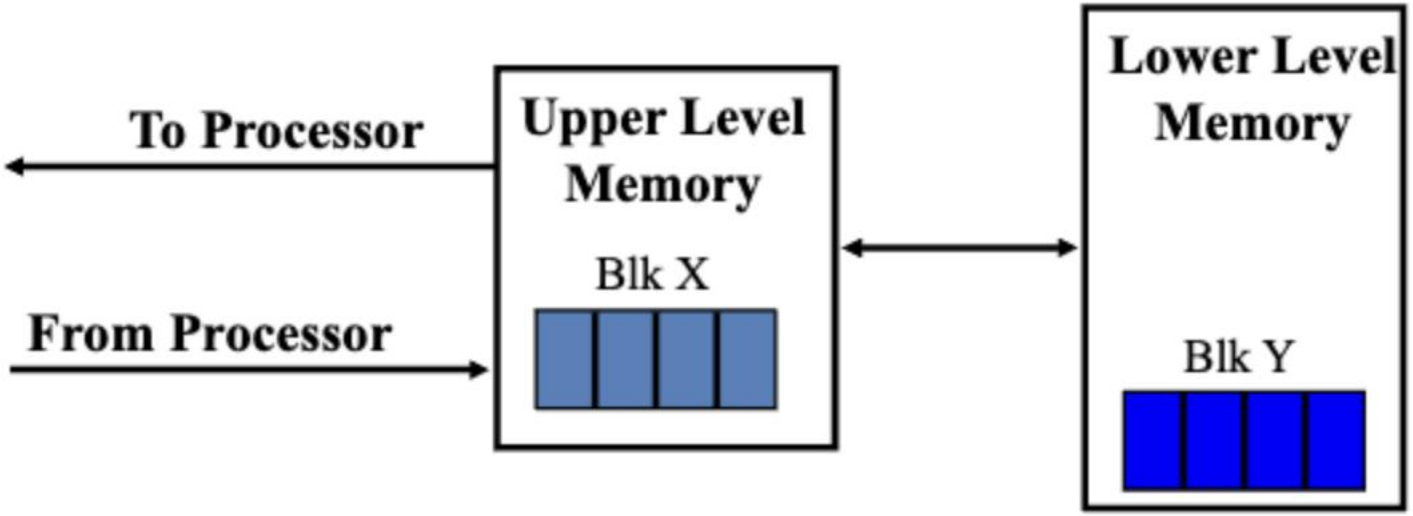

Address
n loop iterations
Instruction fetches
subroutine call
subroutine return
Stack accesses
argument access
vector access
scalar accesses
Data accesses
Time

局部性原理

- **程序访问的局部性原理：对于绝大多数程序来说，程序所访问的指令和数据在地址上不是均匀分布的，而是相对簇聚的**
  - **时间局部性**：程序马上将要用到的信息很可能就是现在正在使用的信息。
    - ✓ 解决方法：多级层次结构。
  - **空间局部性**：程序马上将要用到的信息很可能与现在正在使用的信息在存储空间上是相邻的。
    - ✓ 解决办法：块组织，预取技术。

局部性原理

- **时间局部性**：保持最近访问的数据项最接近处理器
- **空间局部性**：以由地址连续的若干个字构成的块为单位，从低层复制到上一层

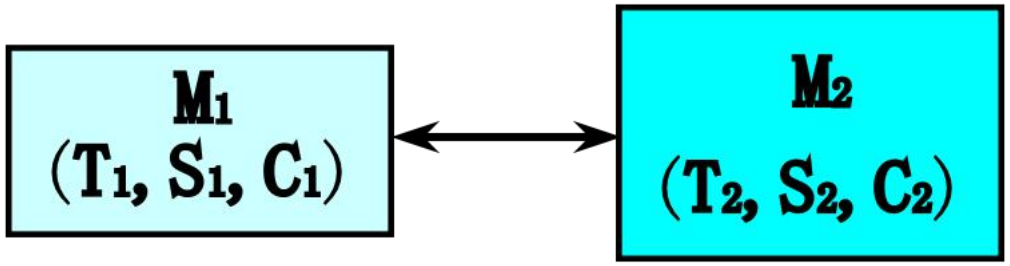


存储层次结构涉及的基本概念

- **块（Block）**
  - 不同层次的Block大小可能不同
  - 命中与命中率
  - 失效与失效率
- **镜像和一致性问题**
  - 高层存储器是较低层存储器的一个镜像
  - 高层存储器内容的修改必须反映到低层存储器中（数据一致性问题）
- **要求：不同层次上块大小可以不同**
  - 以块为单位进行数据交换
  - 不同级Cache块的大小可能不同
  - 页面（块）大小通常大于Cache块大小
  - 存在地址映射问题
  - 物理地址格式 Block Frame Address + Block Offset

性能参数

- 下面仅考虑由M1和M2构成的两级存储层次
  - M1的参数：S1，T1，C1；
  - M2的参数：S2，T2，C2。



性能参数

- **存储容量S**
  - 一般来说，整个存储系统的容量即是第二级存储器M2的容量，即S=S2
- **每位价格C**

$$C = \frac{C_1 S_1 + C_2 S_2}{S_1 + S_2}$$

当$S_1 << S_2$时，$C \approx C_2$

性能参数

- **命中率H 和不命中率F**
  - 命中率H：CPU访问存储系统时，在M1中找到所需信息的概率：

$$H = \frac{N_1}{N_1 + N_2}$$

N_1 —— 访问M_1的次数

N_2 —— 访问M_2的次数

  - 不命中率 （缺失率）：F = 1 - H

性能参数

- **平均访问时间TA**

$$T_A = HT_1 + (1-H)(T_1 + T_M) = T_1 + (1-H)T_M$$

或 $T_A = T_1 + FT_M$

  - 分两种情况来考虑CPU的一次访存：
    - 当命中时，访问时间即为T1（命中时间）；
    - 当不命中时，情况比较复杂：
      - 不命中开销T_M：从向M2发出访问请求到把整个数据块调入M1中所需的时间。
      - 传送一个信息块所需的时间为：T_B

不命中时的访问时间为：$T_M = T_2 + T_B$

常见的存储层次组织

- 寄存器 <-> 内存
  - 由编译器完成调度
- Cache <-> 内存
  - 由硬件完成调度
- 内存 <-> 磁盘
  - 由硬件和操作系统（虚拟管理）
  - 由程序员（应用程序）完成调度

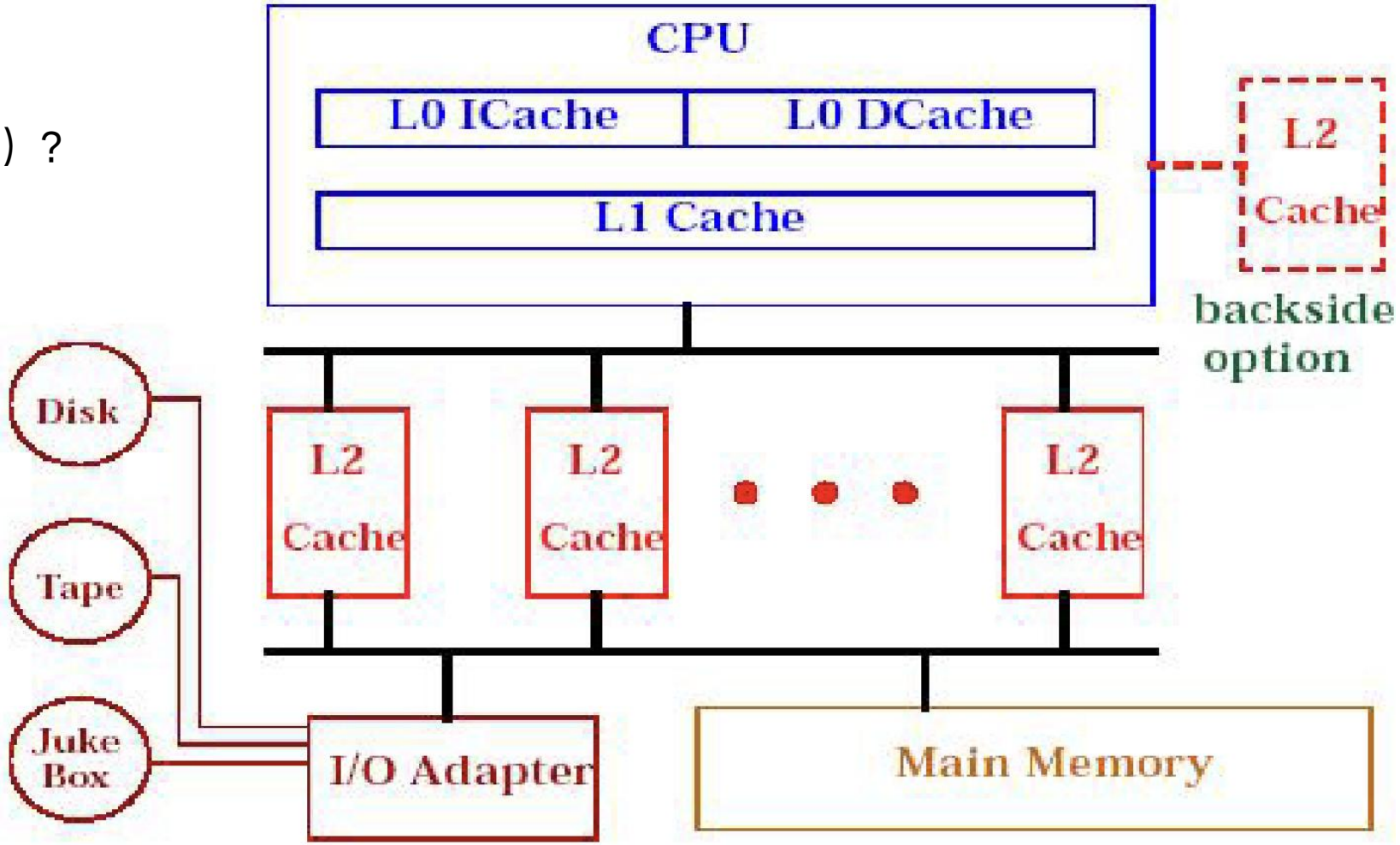
什么是Cache？

- 小而快（SRAM）的存储技术
  - 存储正在访问的部分指令和数据
- 用于减少平均访存时间
  - 通过保持最近访问的数据在处理器附近来挖掘（时间局部性）
  - 通过以块为单位在不同层次移动数据来挖掘（空间局部性）
- 主要目标：
  - 提高访存速度
  - 提高存储容量
  - 降低存储系统成本

内存层次结构样例

Q1: 上下级内存如何映像（映射）？
Q2: 上下级内存如何查找？
Q3: 如何替换？
Q4: 写策略？

MSRW?MSRA?



Q1: 映象规则

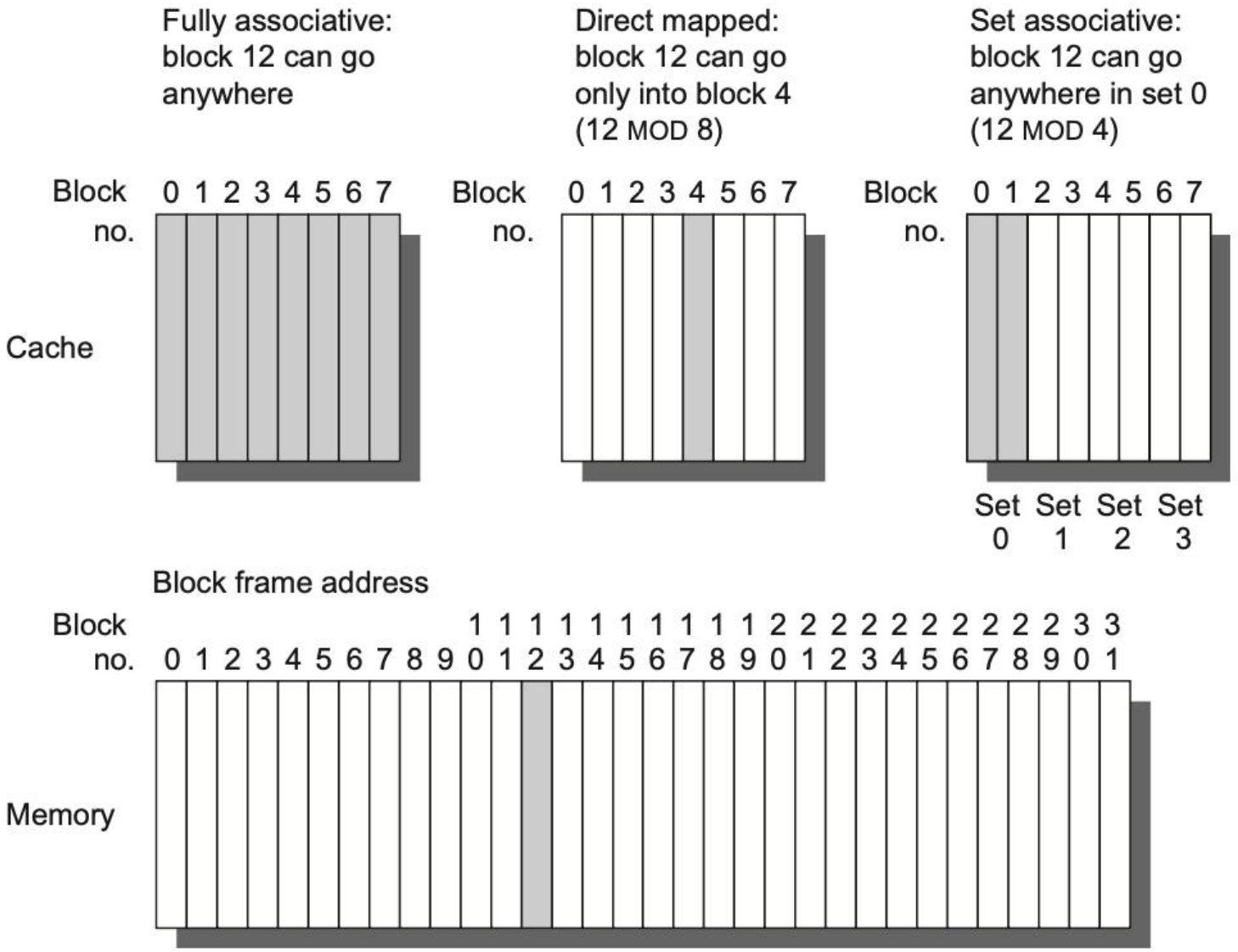
- 当要把一个块从主存调入Cache时，如何放置？
- 三种方式
  - 全相联方式：即所调入的块可以放在Cache中的任何位置
  - 直接映象方式：主存中每一块只能存放在Cache中的唯一位置
    - 一般，主存块地址i与Cache中块地址j的关系为：

$$j = i \bmod M \text{，} M\text{为Cache中的块数}$$

  - 组相联映象：主存中每一块可以被放置在Cache中唯一的一个组中的任意一个位置，组由若干块构成，若一组由n块构成，我们称N路组相联
    - 组间直接映象
    - 组内全相联
    - 若Cache中有G组，则主存中的第i块的组号K：$K = i \bmod (G)$

映象规则 – 举例

- Block 12 放置在 8 Block Cache 中



映象规则讨论

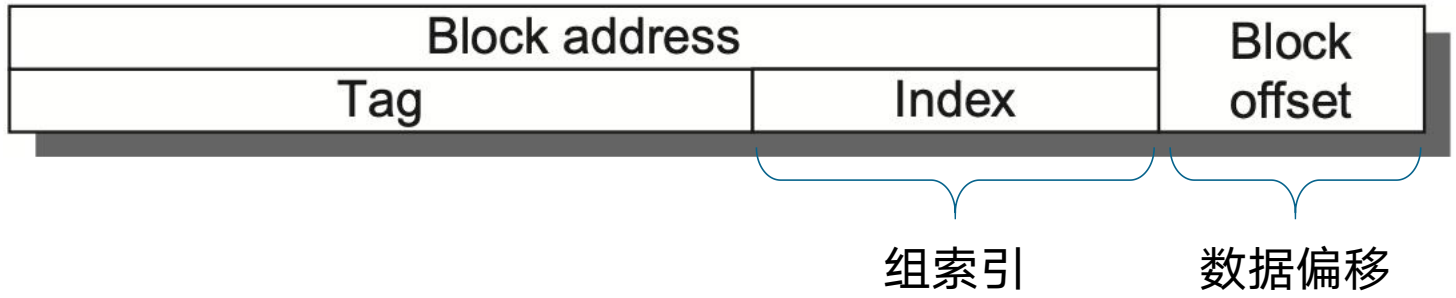
- N-Way组相联：如果每组由N个块构成，Cache的块数为M，则Cache的组数G为M/N
- 不同相联度下的路数和组数

	路数	组数
直接相联	1	M
全相联	M	1
组相联	1< N < M	1 < G < M

- 相联度越高（N值越大），Cache空间利用率就越高，块冲突概率就越小，失效率就越低 
- 相联度越高（N值越大），Cache的实现就越复杂，代价越大 
- 现代大多数计算机都采用直接映象，两路或四路组相联

Q2: 查找方法

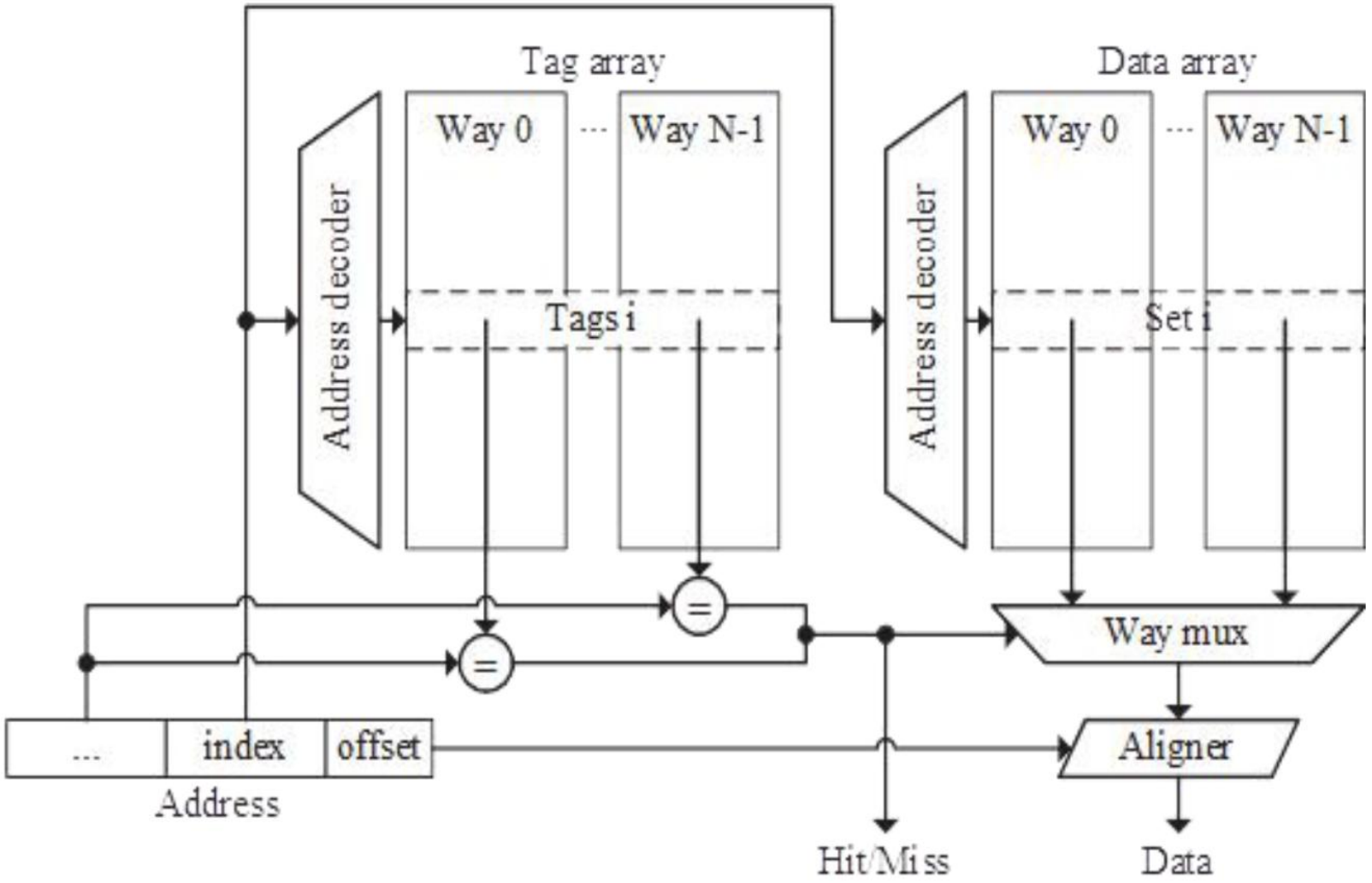
- 在Cache中每一Block都带有Tag域（标记域）
  - Address Tags：标记所访问的单元在哪一块中，这样物理地址就分为三部分： Address Tags ## Block index## block Offset
  - 全相联映象时，没有Block Index
  - 显然 Address tag越短，查找所需代价就越小
  - Status Tags：标记该块的状态，如Valid, Dirty等



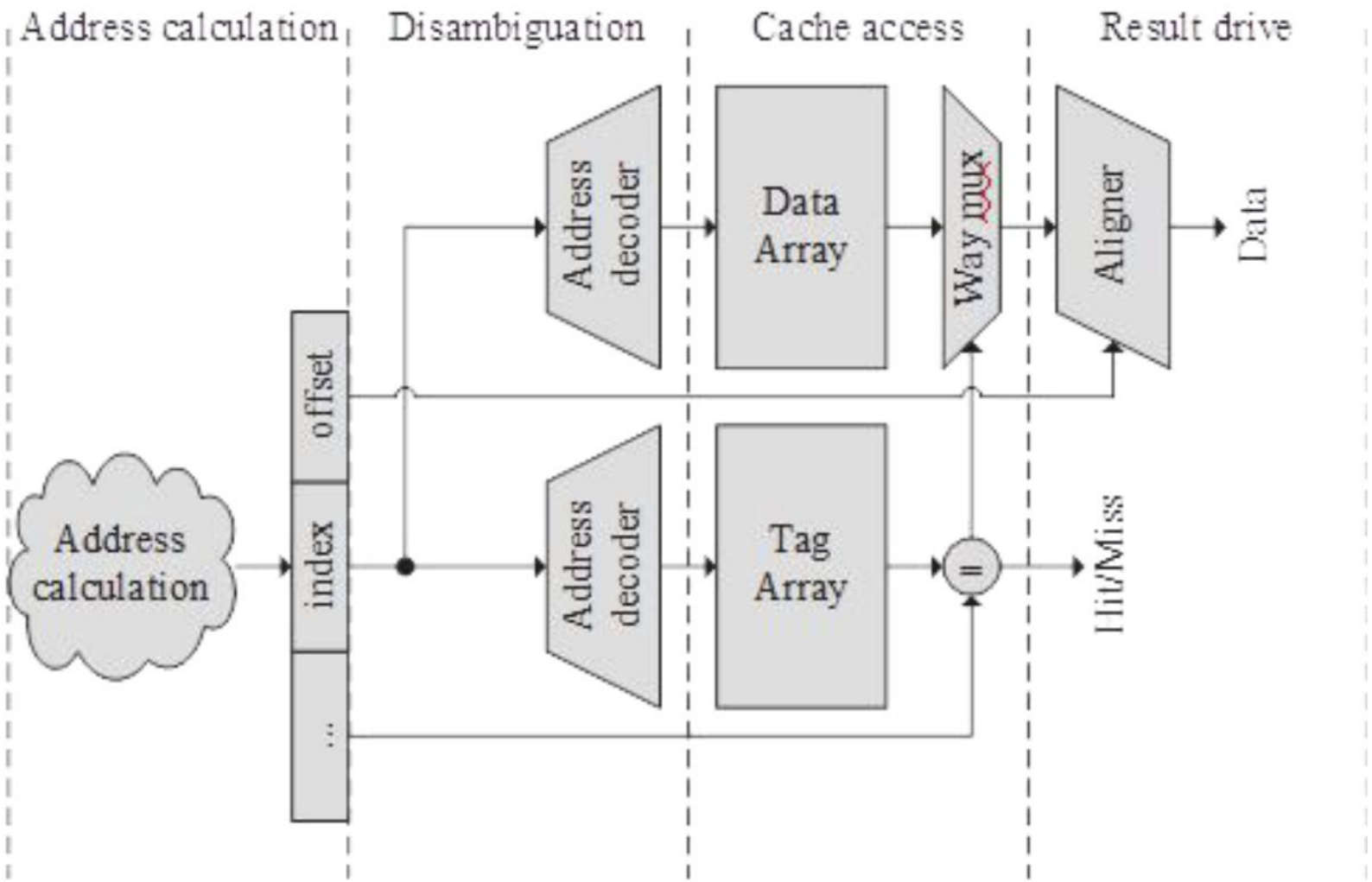
Q2: 查找方法

- 原则：所有可能的标记并行查找，Cache的速度至关重要，即并行查找
- 并行查找的方法
  - 用相联存储器实现，按内容检索
  - 用单体多字存储器和比较器实现
- 显然相联度N越大，实现查找的机制就越复杂，代价就越高
- 无论直接映象还是组相联，查找时，只需比较Tag，Index无需参加比较

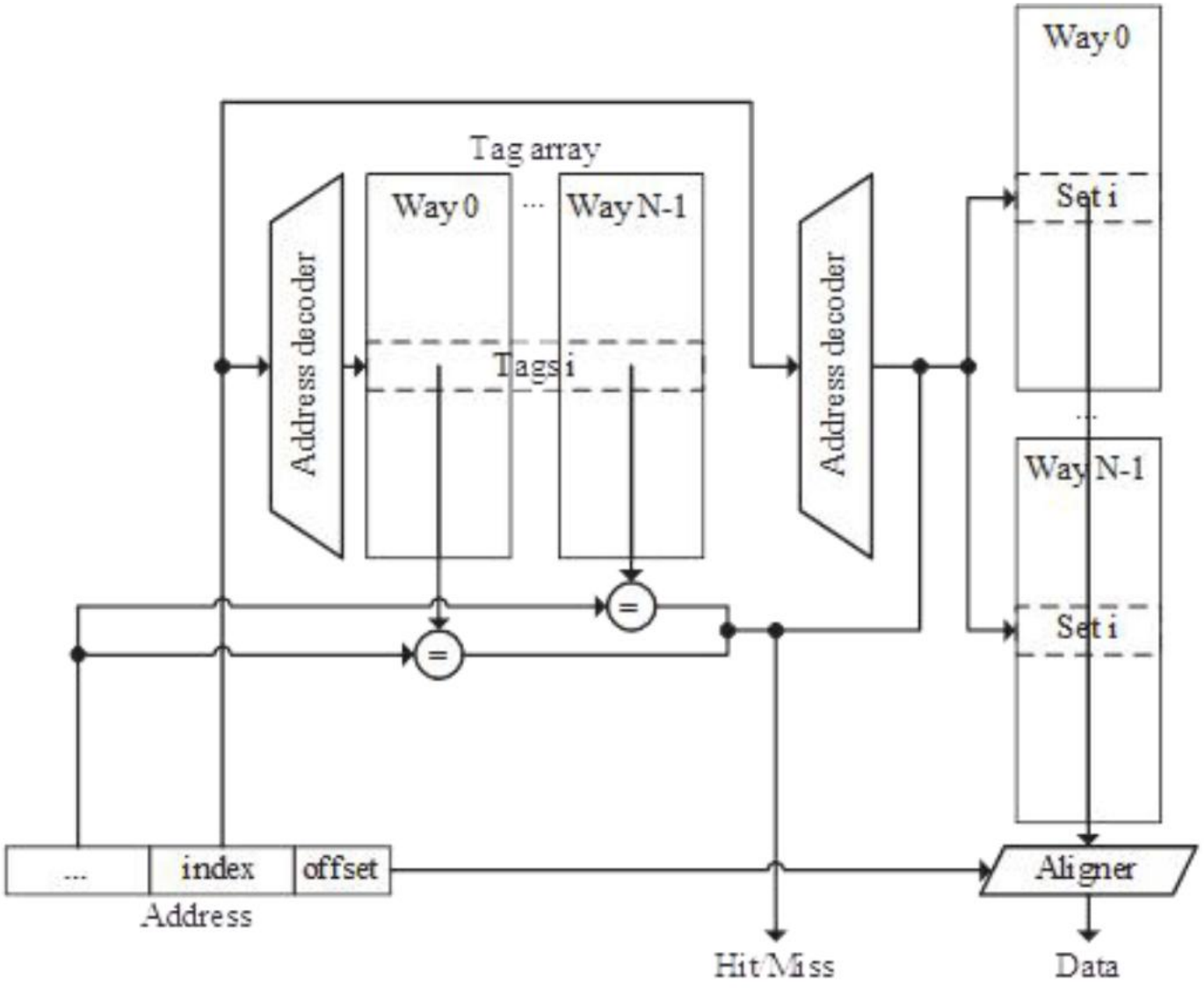
Tag和数据阵列并行访问的逻辑结构



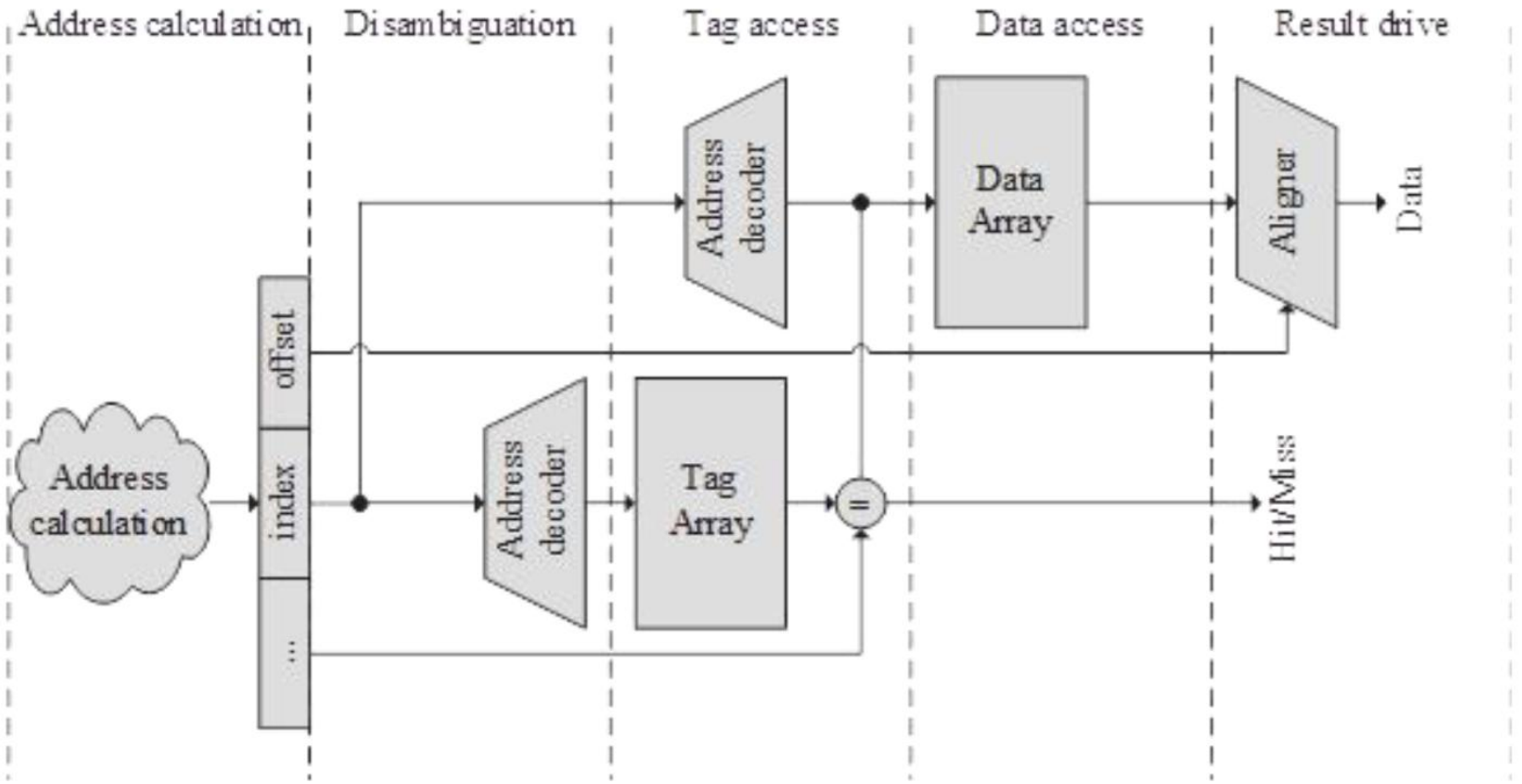
Tag和数据阵列并行访问的流水线模式



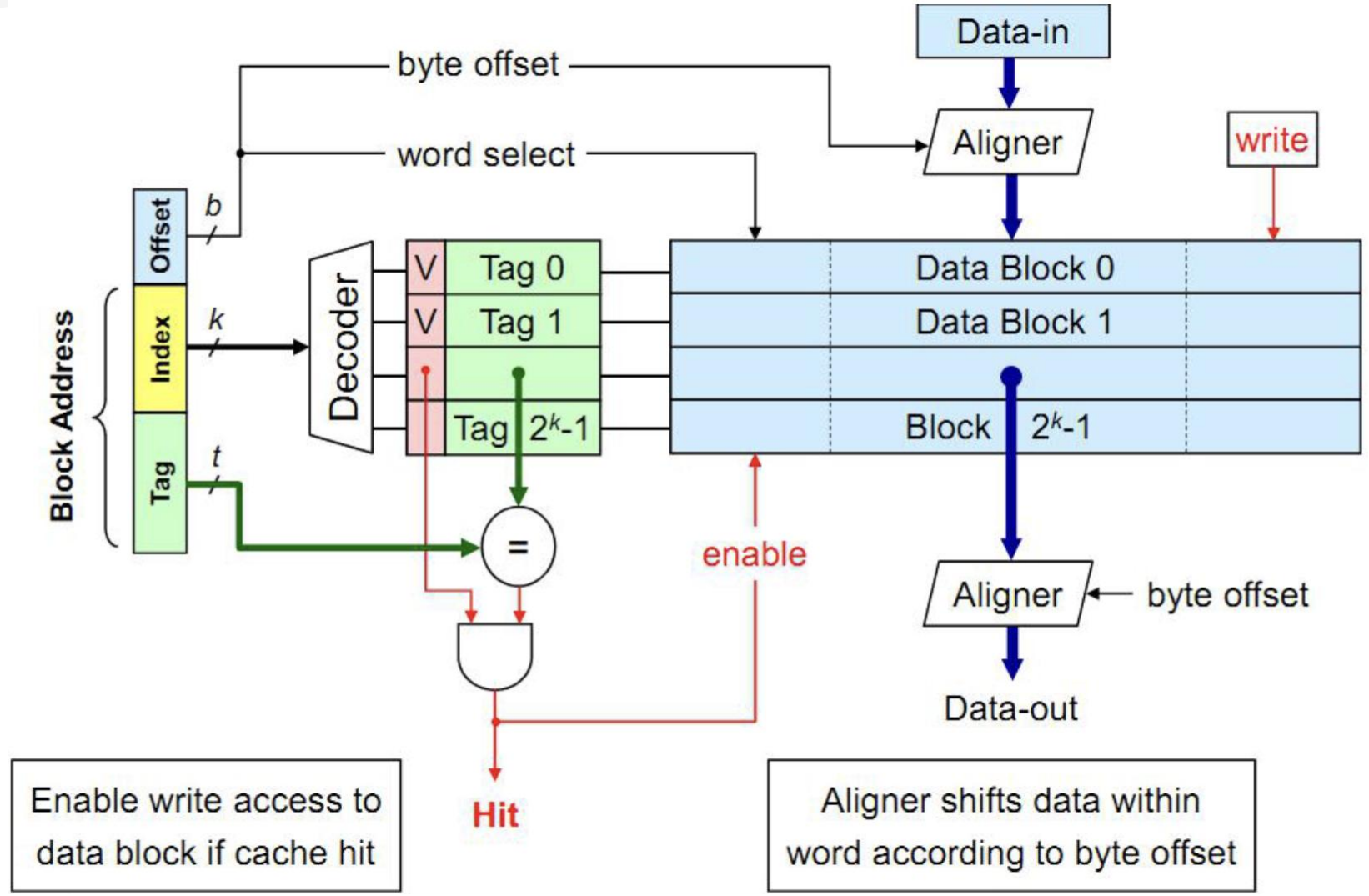
Tag和数据阵列串行访问的逻辑结构



Tag 和数据阵列串行访问的流水线模式

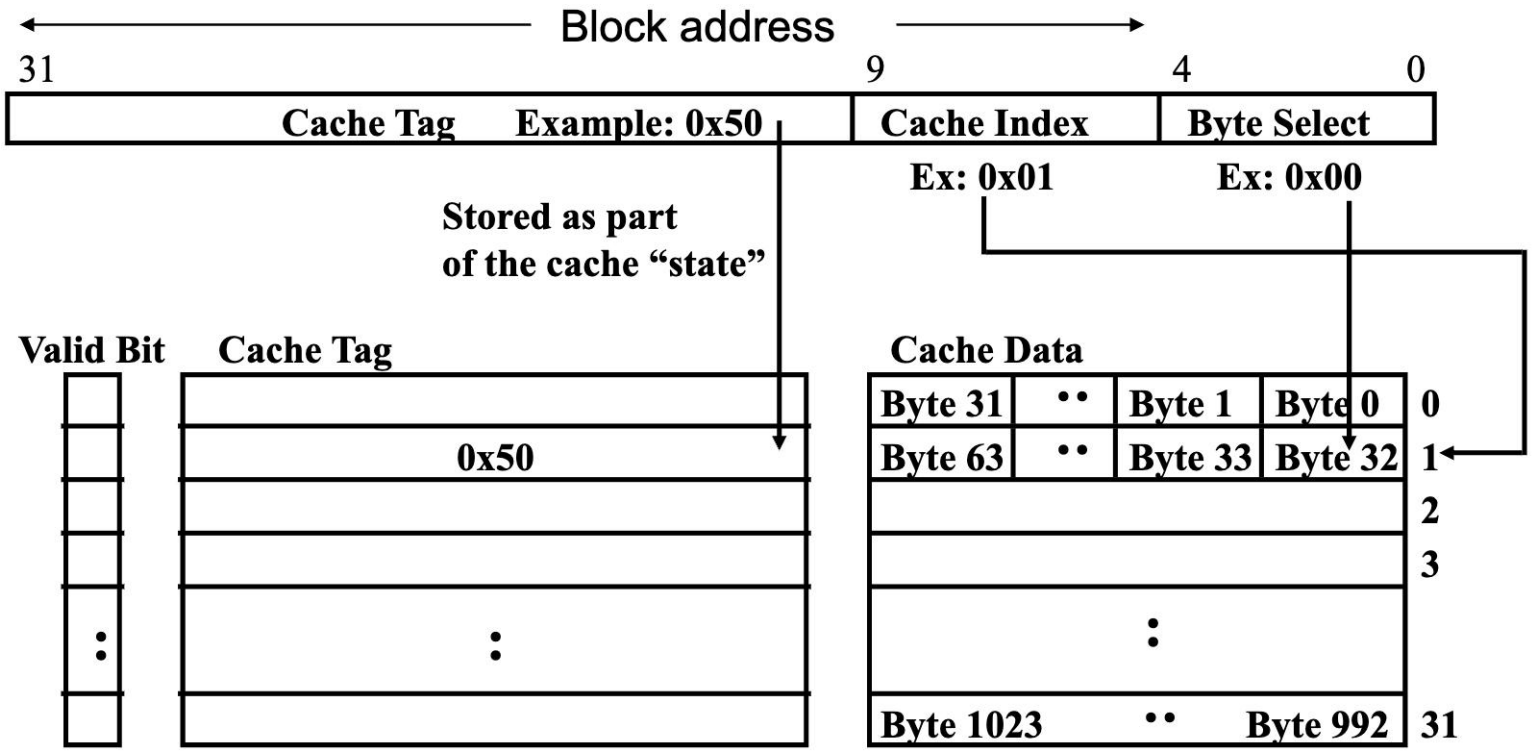


直接相联Cache查找过程



例子：1KB直接相联（32B Blocks）

- 对于容量为 2^N 字节的cache:
  - 最高(32-N)位部分 为 Cache Tag
  - 最低M位为字节选择位 (Block Size = 2^M)



直接相联Cache查找过程（例）

- 举例：检查是否命中

t=1	s=2	b=1
x	xx	x

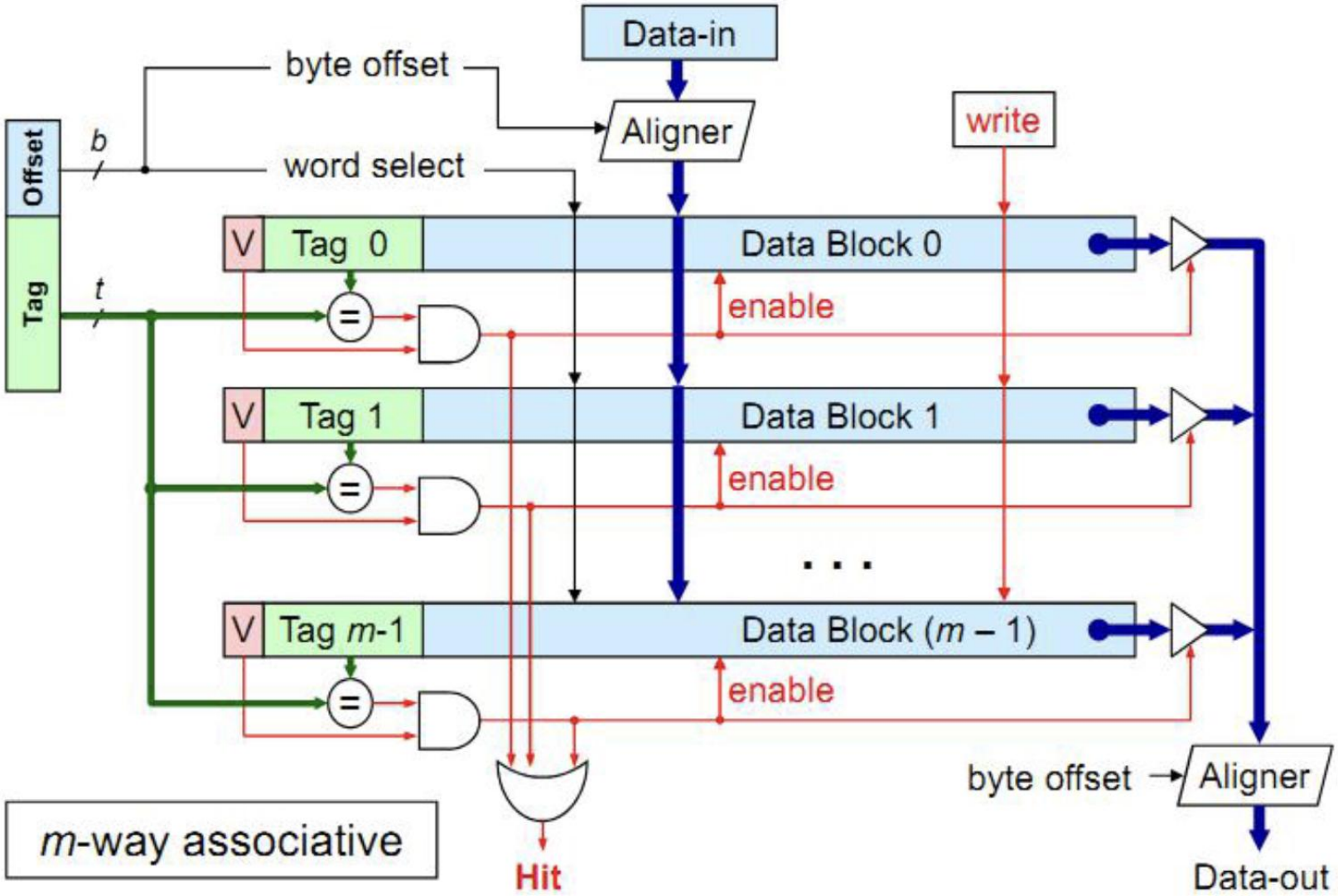
M=16 bytes (4-bit addresses), B=2 bytes/block,
S=4 sets, E=1 Blocks/set

	v	Tag	Block
Set 0	1	0	M[0-1]
Set 1			
Set 2			
Set 3	1	0	M[6-7]

Address trace (reads, one byte per read):

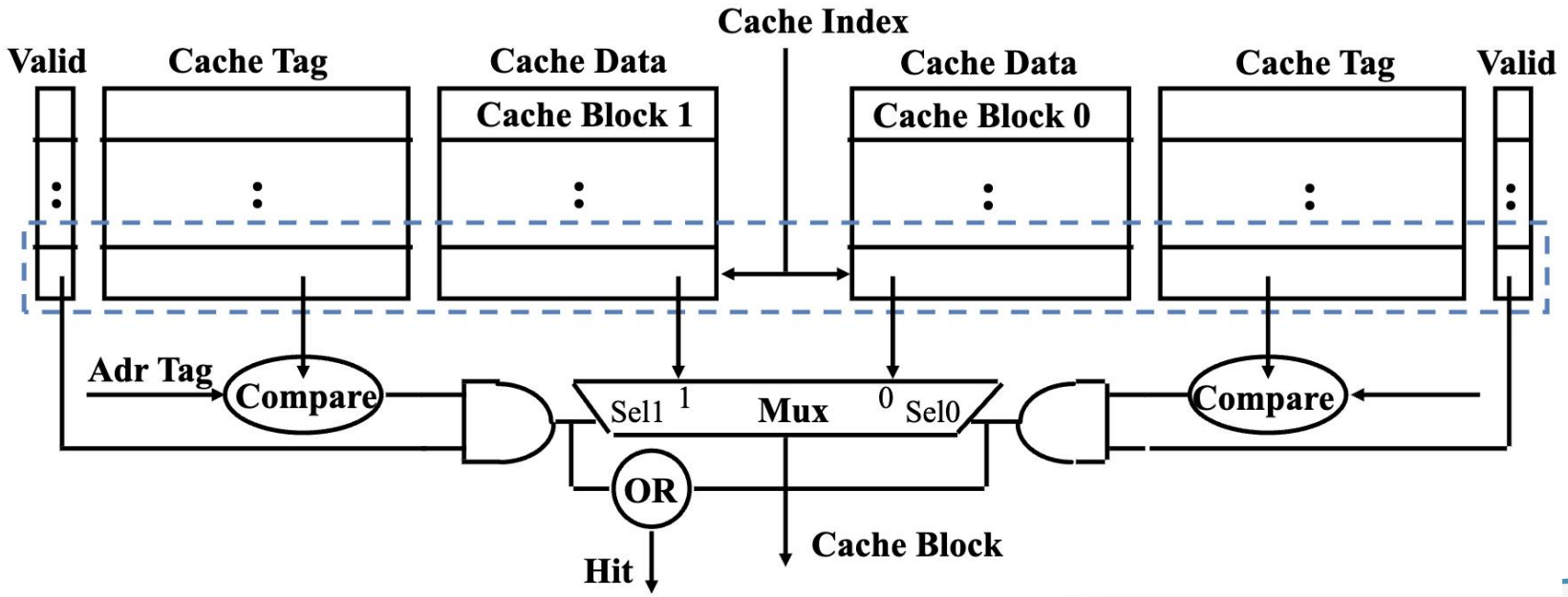
0	[$0\underline{00}0_2$],	miss
1	[$0\underline{00}1_2$],	hit
7	[$0\underline{11}1_2$],	miss
8	[$1\underline{00}0_2$],	miss
0	[$0\underline{00}0_2$]	miss

全相联Cache查找过程



例子：组相联缓存

- N路组相联: 每一个Cache索引对应N个Cache entries
  - 这N个Cache项并行操作
- 举例: 2路组相联Cache
  - Cache index 选择Cache中的一组
  - 这一组中的两块对应的Tags与输入的地址同时比较
  - 根据比较结果选择数据



2路-组相联Cache查找过程（例）

➢ 举例：检查是否命中

t=2	s=1	b=1
xx	x	x

M=16 bytes (4-bit addresses), B=2 bytes/block,
S=2 sets, E=2 Blocks/set

	v	Tag	Block
Set 0	1	00	M[0-1]
	1	10	M[8-9]
Set 1	1	01	M[6-7]

Address trace (reads, one byte per read):

0	[00$\underline{0}$0$_2$],	miss
1	[00$\underline{0}$1$_2$],	hit
7	[01$\underline{1}$1$_2$],	miss
8	[10$\underline{0}$0$_2$],	miss
0	[00$\underline{0}$0$_2$]	hit

Q3: 替换算法

- 主存中块数一般比Cache中的块多，可能出现该块所对应的一组或一个Cache块已全部被占用的情况，这时需强制腾出其中的某一块，以接纳新调入的块，替换哪一块，这是替换算法要解决的问题：
  - 直接映象，因为只有一块，别无选择
  - 组相联和全相联有多种选择
- 替换方法
  - 随机法（Random)，随机选择一块替换
    - 优点：简单，易于实现
    - 缺点：没有考虑Cache块的使用历史，反映程序的局部性较差，失效率较高
  - FIFO - 选择最早调入的块
    - 优点：简单
    - 虽然利用了同一组中各块进入Cache的顺序，但还是反映程序局部性不够，因为最先进入的块，很可能是经常使用的块
  - 最近最少使用法LRU (Least Recently Used)
    - 优点：较好的利用了程序的局部性，失效率较低
    - 缺点：比较复杂，硬件实现较困难

LRU和Random的比较（失效率）

	Associativity								
	Two-way			Four-way			Eight-way		
Size	LRU	Random	FIFO	LRU	Random	FIFO	LRU	Random	FIFO
16 KiB	114.1	117.3	115.5	111.7	115.1	113.3	109.0	111.8	110.4
64 KiB	103.4	104.3	103.9	102.4	102.3	103.1	99.7	100.5	100.3
256 KiB	92.2	92.1	92.5	92.1	92.1	92.5	92.1	92.1	92.5

Data Cache misses per 1000 instructions comparing LRU, Random, FIFO replacement for several sizes and associativities.
These data were collected for a block size of 64 bytes for the Alpha architecture using 10 SPEC2000 benchmarks. Five are from SPECint2000(gap, gcc, gzip, mcf and perl) and five are from SPECfp2000(applu, art, equake, lucas and swim)

- 观察结果（失效率）
  - 相联度高，失效率较低。
  - Cache容量较大，失效率较低。
  - LRU 在Cache容量较小时，失效率较低
  - 随着Cache容量的加大，Random的失效率在降低

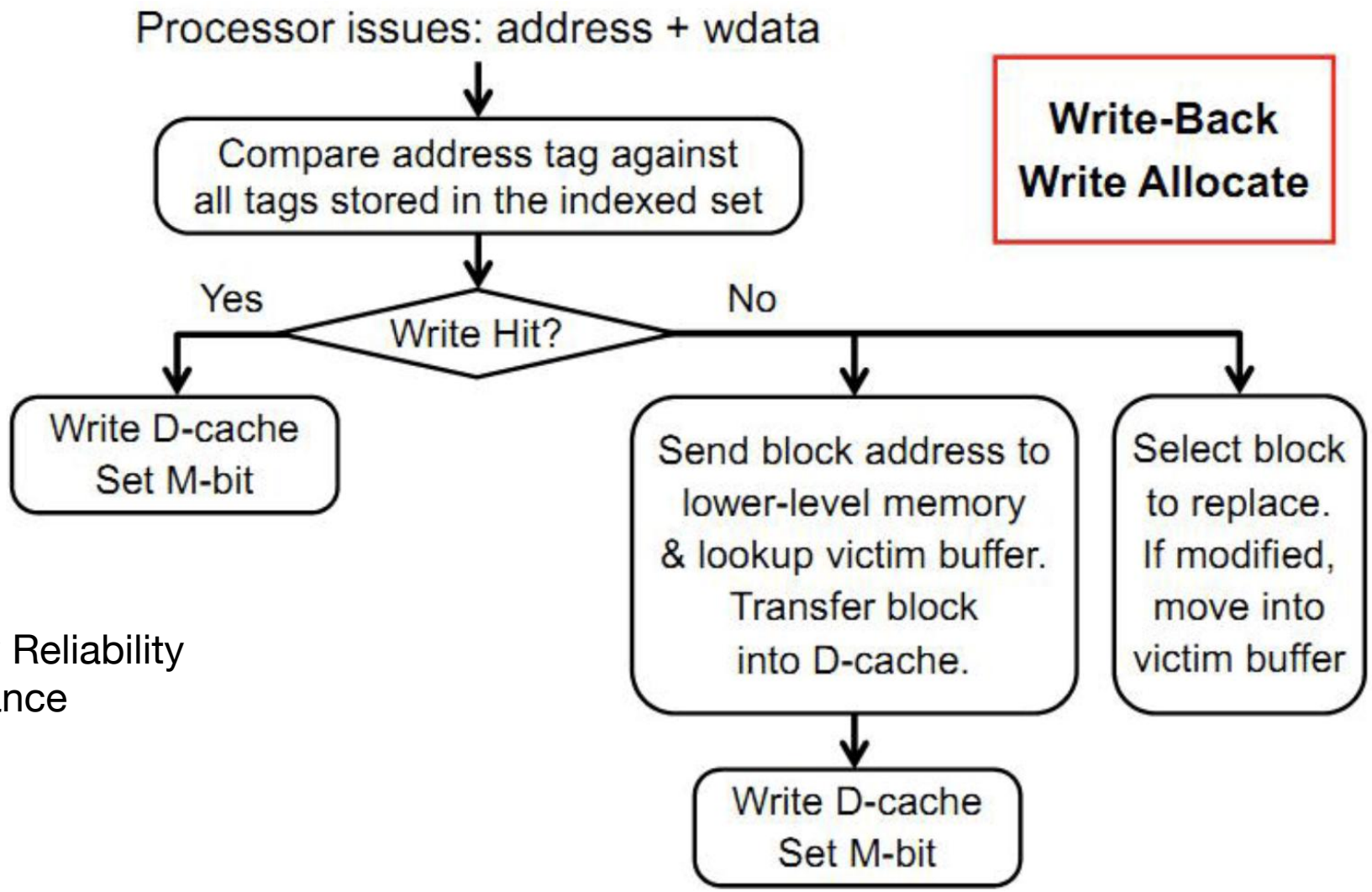
Q4: 写策略

- 程序对存储器读操作占26%， 写操作占9%
  - 占访问数据Cache的比例：9/(26+9) 大约为25%
- 大概率事件优先原则 - 优化Cache的读操作
- Amdahl定律：不可忽视“写”的速度
- “写”的问题
  - 读出标识，确认命中后，对Cache写 （串行操作）
  - Cache与主存内容的一致性问题
- 写策略就是要解决: 何时更新主存问题

两种写策略：写直达和写回

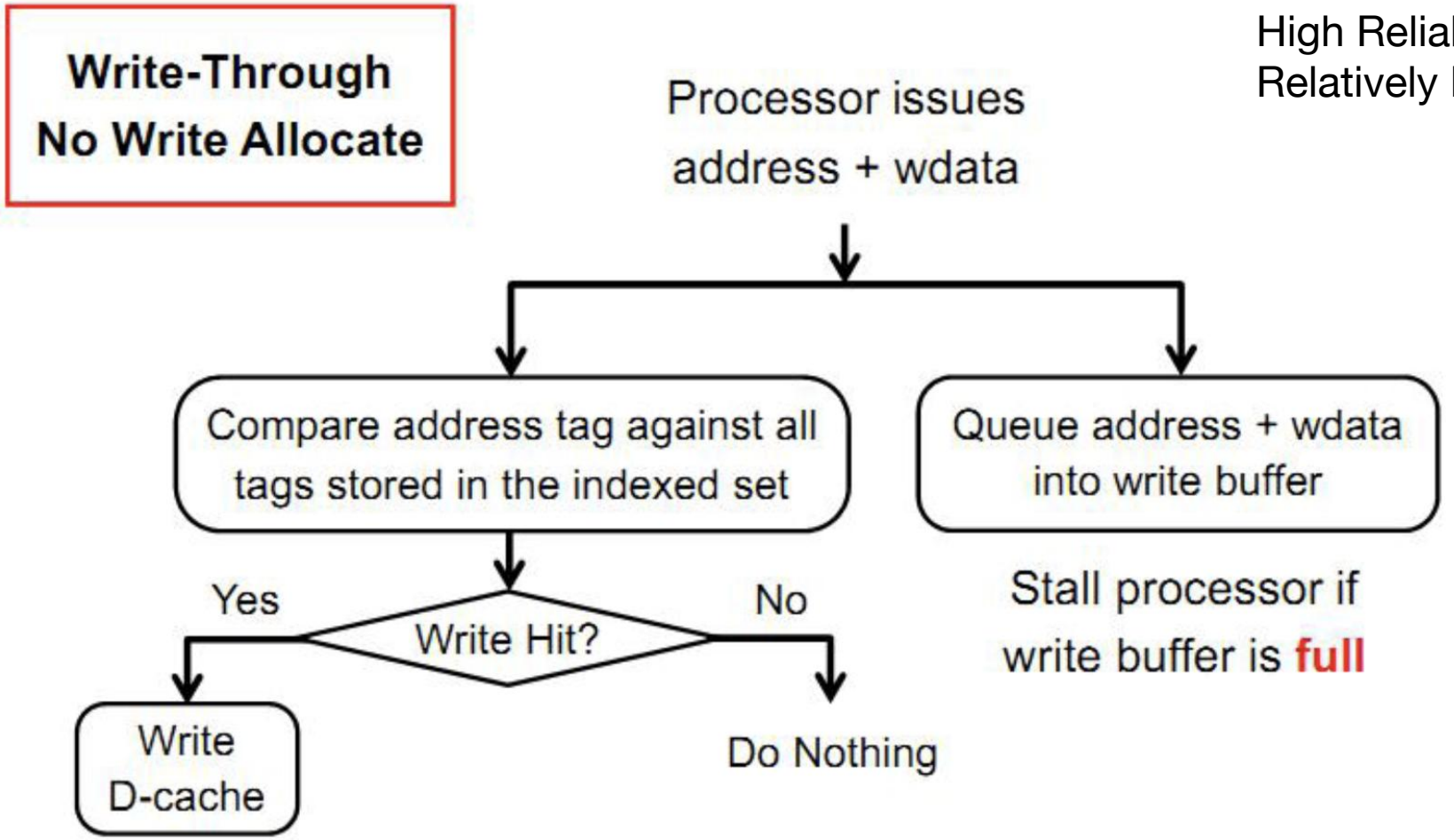
- 写直达法（Write through)
  - 优点：易于实现，容易保持不同层次间的一致性
  - 缺点：速度较慢
- 写回法（Write Back）
  - 优点：速度快，减少访存次数
  - 缺点：一致性问题
- 当发生写失效时的两种策略
  - 按写分配法(Write allocate)：写失效时，先把所写单元所在块调入Cache，然后再进行写入，也称写时取（Fetch on Write)方法
  - 不按写分配法（No-write allocate): 写失效时，直接写入下一级存储器，而不将相应块调入Cache，也称绕写法（Write around)
  - 原则上以上两种方法都可以应用于写直达法和写回法，一般情况下
    - Write Back用Write allocate
    - Write Through用No-write allocate

写回与写分配



Relatively Low Reliability
High Performance

写直达与不按写分配

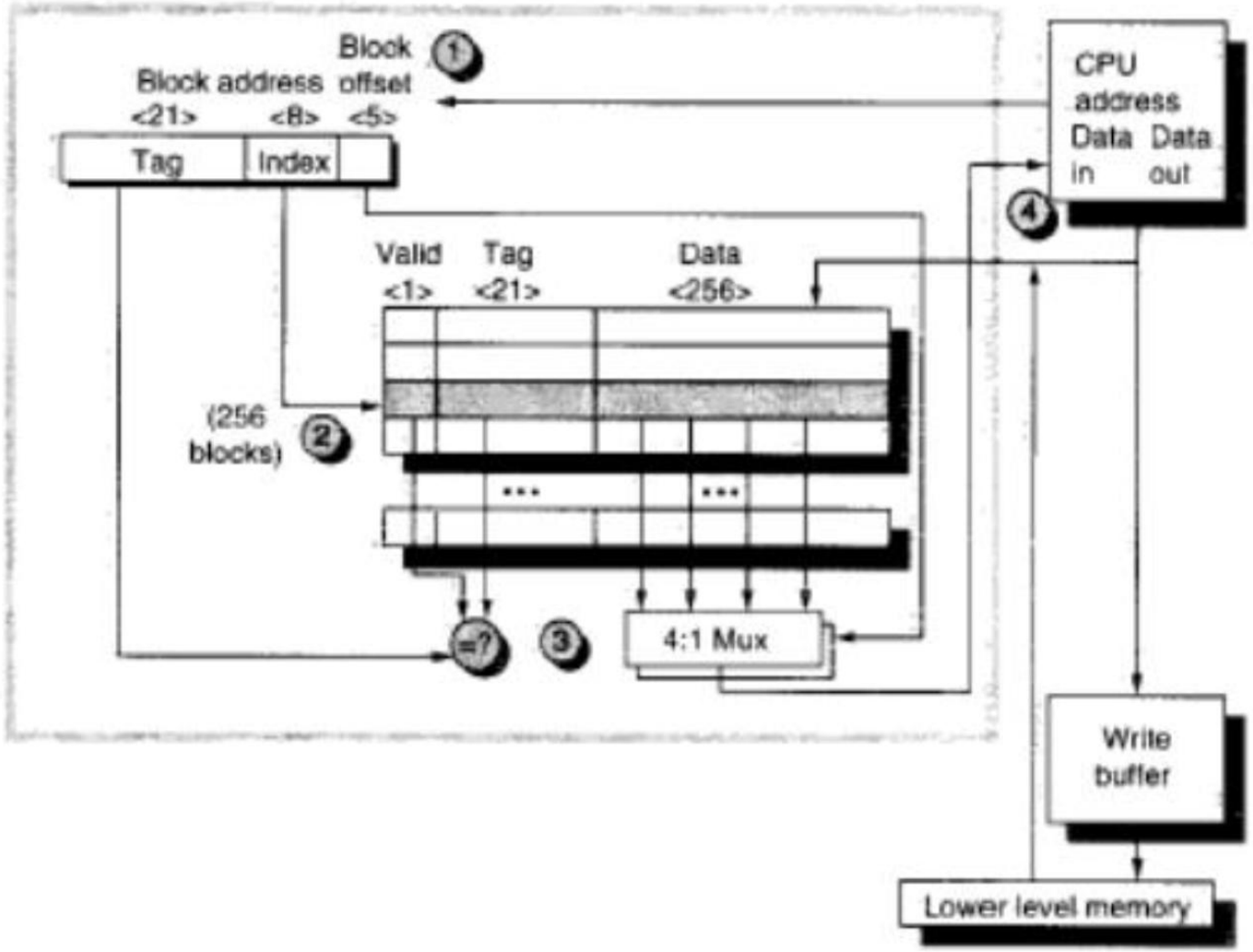


High Reliability
Relatively Low Performance

实例：Alpha AX 21064 Cache结构（数据Cache)

- 基本技术特性
  - 容量 8KB ，Block 32Bytes, 共256个Blocks，每个字为8个字节
  - 直接映象
  - 写直达法，写失效时，no-write allocate 方法
  - 写缓冲：4个blocks
- 21064物理地址34位
  - 21位tag##8位index ##5位块内偏移
- Cache命中的步骤
  - 读命中
  - 写命中
- Cache失效
  - Cache向CPU发暂停信号
  - 块传送，21064 Cache与下一级存储器之间数据通路16字节，传送全部32字节需要10个cycles

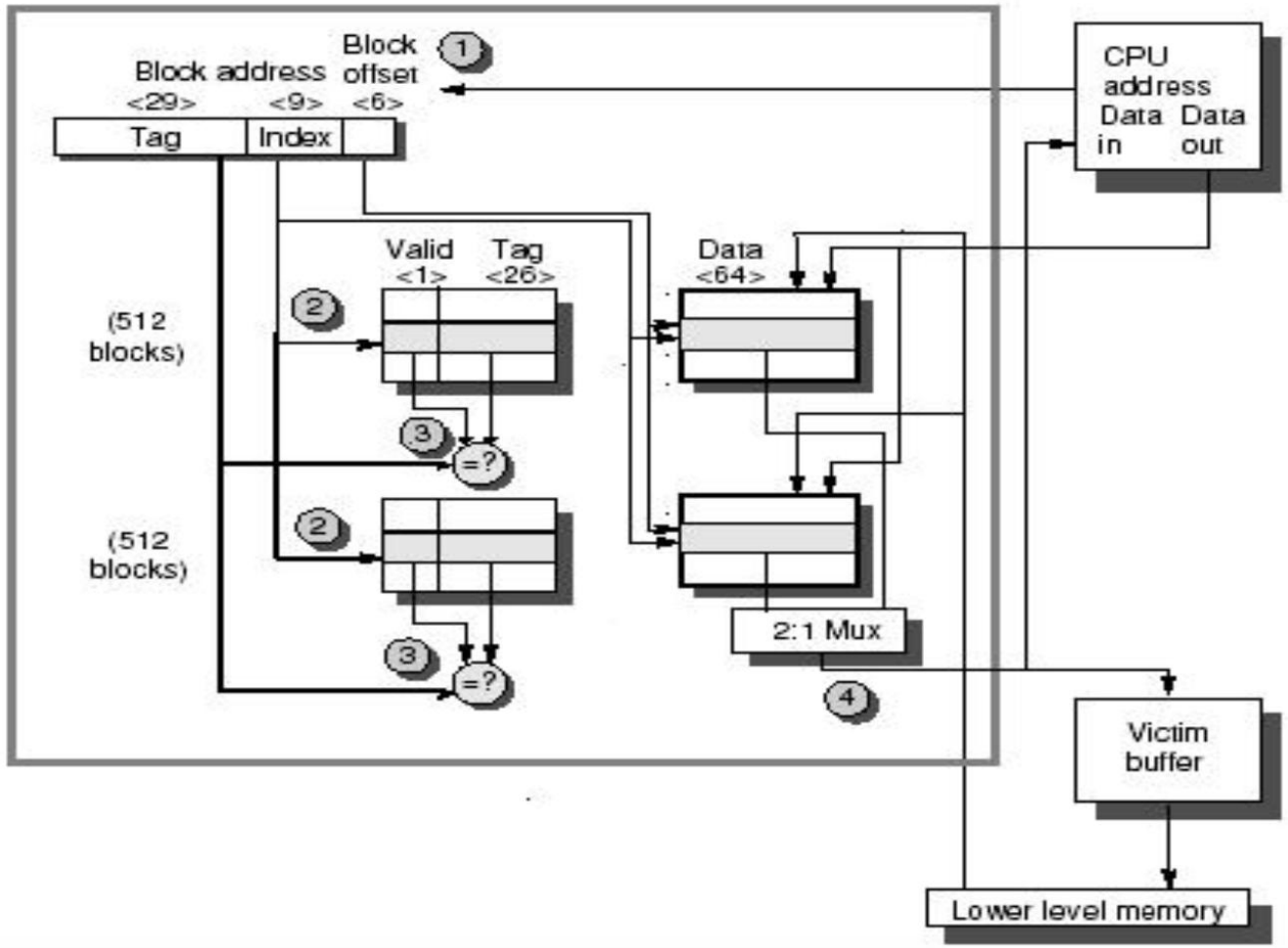
实例：Alpha AX 21064 Cache结构（数据Cache)



实例：Alpha AX 21264 Cache结构（数据Cache)

- 基本技术特征
  - Cache size: 64Kbytes
  - Block size: 64-bytes
  - Two-way 组相联
  - Write back, 写失效时，write allocate
- 21264 48-bits 虚拟地址，虚实映射为44bits的物理地址，也支持43-bits虚拟地址，虚实映射为41-bits的物理地址
  - 29位tags##9位index##6位 Block offset

实例：Alpha AX 21264 Cache结构（数据Cache)



Cache的基本概念小结

- 存储系统分层结构
  - 为什么要这样做？
  - 为什么可以这样做？
- 存储系统的性能指标：速度、容量和价格
  - 平均访存时间=命中时间+失效率×失效开销
- Cache 4Q
  - 映射规则
  - 查找方法
  - 替换策略
  - 写策略

目录



➢ Cache的基本概念

- 存储系统的层次结构
- Cache基本知识

➢ Cache的基本优化方法

- Cache性能分析
- 降低失效率
- 减少失效开销
- 缩短命中时间

Cache性能分析

➢平均访存时间（AMAT模型）

平均访存时间 = 命中时间 + 不命中率×不命中开销

$$AMAT = T_{hit} + F \times T_{miss_penalty}$$

- ✓ 可以采用绝对时间——如一次命中需0.25-1.0纳秒；
- ✓ 或是用CPU等待存储器的时钟周期数——如缺失代价用35-100个时钟周期表示；
- ✓ 仍旧不能代替执行时间。

Cache性能分析

➢ 程序执行时间

CPU时间 = CPI × 时钟周期时间

= （CPU执行周期数+存储器停顿周期数）× 时钟周期时间

- ✓ 存储器停顿时钟周期数 = “读”的次数 × 读不命中率 × 读不命中开销 + “写”的次数 × 写不命中率 × 写不命中开销
- ✓ 存储器停顿时钟周期数 = 访存次数 × 不命中率 × 不命中开销

$$CPU\text{时间} = (\text{CPU执行周期数} + \text{访存次数} \times \text{不命中率} \times \text{不命中开销}) \times \text{时钟周期时间}$$

$$CPU\text{时间} = IC \times \left(CPI_{\text{execution}} + \frac{\text{访存次数}}{\text{指令总数}} \times \text{不命中率} \times \text{不命中开销}\right) \times \text{时钟周期时间}$$

Cache性能分析举例

- 例：用一个和Alpha AXP类似的机器作为第一个例子。假设Cache不命中开销为50个时钟周期，当不考虑存储器停顿时，所有指令的执行时间都是2.0个时钟周期，访问Cache不命中率为2%，平均每条指令访存1.33次。试分析Cache对性能的影响。

(1) 考虑Cache的不命中后，性能为：

$$\text{CPU时间}_{\text{有cache}} = IC \times \left(CPI_{execution} + \text{每条指令的平均访存次数} \times \text{不命中率} \times \text{不命中开销}\right) \times \text{时钟周期时间}$$

$$= IC \times \left(2.0 + 1.33 \times 2\,\% \times 50\right) \times \text{时钟周期时间}$$

$$= IC \times 3.33 \times \text{时钟周期时间}$$

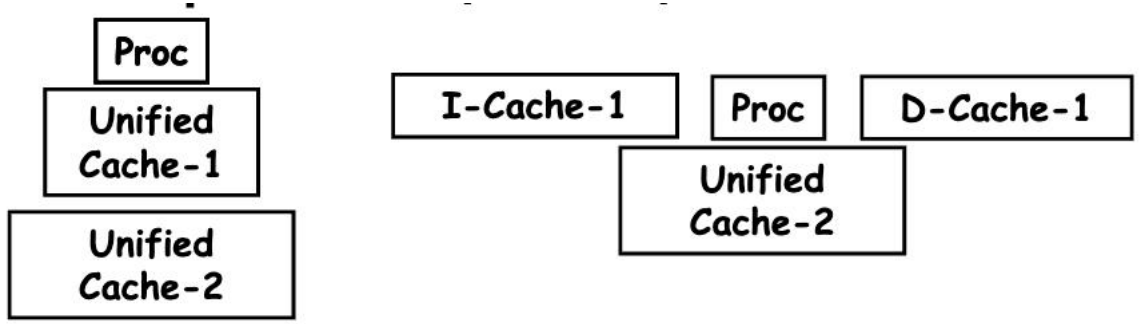
实际CPI：3.33，则3.33/2.0 = 1.63(倍)，即CPU时间也增加为原来的1.63倍。

(2) 但若不采用Cache,则：

$$CPI = 2.0 + 50 \times 1.33 = 68.5$$

Harvard Architecture vs. Unified I&D

- Unified vs Separate I&D (Harvard)



- Statistics (given in H&P):
  - 16KB I&D: Inst miss rate=0.64%, Data miss rate=6.47%
  - 32KB unified: Aggregate miss rate=1.99%
- Which is better (ignore L2 cache)?
  - Assume 33% data ops -> 75% accesses from instructions (1.0/1.33)
  - hit time=1, miss time=50
  - Note that data hit has 1 stall for unified cache (only one port)
- AMATHarvard=75%x(1+0.64%x50)+25%x(1+6.47%x50) = 2.05
- AMATUnified =75%x(1+1.99%x50)+25%x(1+1+1.99%x50)= 2.24

UltraSPARC III例子

- 以顺序执行的计算机 UltraSPARC III为例，假设Cache失效开销为100 clock cycles，所有指令忽略存储器停顿需要1个cycle，考虑以下两种Cache失效方式：
  - (1)假设平均失效率为2%，平均每条指令访存1.5次
  - **(2)假设每1000条指令cache失效次数为30次**

  分别基于上述两种条件计算处理器的性能
- 结论:

  CPUtime=IC*(1+2%*1.5*100)*T=IC*4*T
  - CPI越低，固定周期数的Cache失效开销的相对影响就越大
  - 在计算CPI时，失效开销的单位是时钟周期数。因此，即使两台计算机的存储层次完全相同， 时钟频率较高的CPU的失效开销会较大，其CPI中存储器停顿部分也就较大。

Cache对于低CPI，高时钟频率的CPU来说更加重要

考虑不同组织结构的Cache对性能的影响

- 例题：直接映像Cache和两路组相联Cache，试问他们对CPU性能的影响?先求平均访存时间，然后再计算CPU性能。

  分析时请用以下假设:

  - (1)理想Cache(命中率为100%)情况下CPI 为2.0，时钟周期为2ns，平均每条指令访存1.3次
  - (2)两种Cache容量均为64KB，块大小都是32B
  - (3)采用组相联时，由于多路选择器的存在，时钟周期增加到原来的1.1倍（命中缓存开销）
  - (4)两种结构的失效开销都是70ns (在实际应用中，应取整为整数个时钟周期)
  - (5)命中时间为1个cycle，64KB直接映像Cache的失效率为1.4%, 相同容量的两路组相联Cache的失效率为1.0%

Cache性能公式总结

$$2^{\text{index}} = \frac{\text{Cache size}}{\text{Block size} \times \text{Set associativity}}$$

$$\text{CPU execution time} = (\text{CPU clock cycles} + \text{Memory stall cycles}) \times \text{Clock cycle time}$$

$$\text{Memory stall cycles} = \text{Number of misses} \times \text{Miss penalty}$$

$$\text{Memory stall cycles} = \text{IC} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

$$\frac{\text{Misses}}{\text{Instruction}} = \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}}$$

$$\text{Average memory access time} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

$$\text{CPU execution time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \frac{\text{Memory stall clock cycles}}{\text{Instruction}} \right) \times \text{Clock cycle time}$$

$$\text{CPU execution time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty} \right) \times \text{Clock cycle time}$$

$$\text{CPU execution time} = \text{IC} \times \left(\text{CPI}_{\text{execution}} + \text{Miss rate} \times \frac{\text{Memory accesses}}{\text{Instruction}} \times \text{Miss penalty} \right) \times \text{Clock cycle time}$$

$$\frac{\text{Memory stall cycles}}{\text{Instruction}} = \frac{\text{Misses}}{\text{Instruction}} \times (\text{Total miss latency} - \text{Overlapped miss latency})$$

$$\text{Average memory access time} = \text{Hit time}_{\text{L1}} + \text{Miss rate}_{\text{L1}} \times (\text{Hit time}_{\text{L2}} + \text{Miss rate}_{\text{L2}} \times \text{Miss penalty}_{\text{L2}})$$

$$\frac{\text{Memory stall cycles}}{\text{Instruction}} = \frac{\text{Misses}_{\text{L1}}}{\text{Instruction}} \times \text{Hit time}_{\text{L2}} + \frac{\text{Misses}_{\text{L2}}}{\text{Instruction}} \times \text{Miss penalty}_{\text{L2}}$$

Figure B.7 Summary of performance equations in this appendix. The first equation calculates the cache index size, and the rest help evaluate performance. The final two equations deal with multilevel caches, which are explained early in the next section. They are included here to help make the figure a useful reference.

改进Cache性能的方法

- 平均访存时间=命中时间+失效率×失效开销
- 从上式可知，基本途径
  - 降低失效率
  - 减少失效开销
  - 缩短命中时间

平均访存时间=命中时间+失效率×失效开销

改进Cache性能的方法

- **平均访存时间=命中时间+失效率×失效开销**
- 从上式可知，基本途径
  - 降低失效率
    - 1、增加Cache块的大小
    - 2、增大Cache容量
    - 3、提高相联度
  - 减少失效开销
    - 4、多级Cache
    - 5、使读失效优先于写失效
  - 缩短命中时间
    - 6、避免在索引缓存期间进行地址转换

平均访存时间=命中时间+失效率×失效开销

Cache失效的原因可分为三类 3C

- 强制性失效 (Compulsory)
  - 第一次访问某一块，只能从下一级Load，也称为冷启动或首次访问失效
- 容量失效(Capacity)
  - 如果程序执行时，所需块由于容量不足，不能全部调入Cache， 则当某些块被替换后，若又重新被访问，就会发生失效。
  - 可能会发生“抖动”现象
- 冲突失效(Conflict (collision))
  - 组相联和直接相联的副作用
  - 若太多的块映像到同一组(块)中，则会出现该组中某个块被别的块替换(即使别的组或块有空闲位置)，然后又被重新访问的情况，这就属于冲突失效

各种类型的失效率

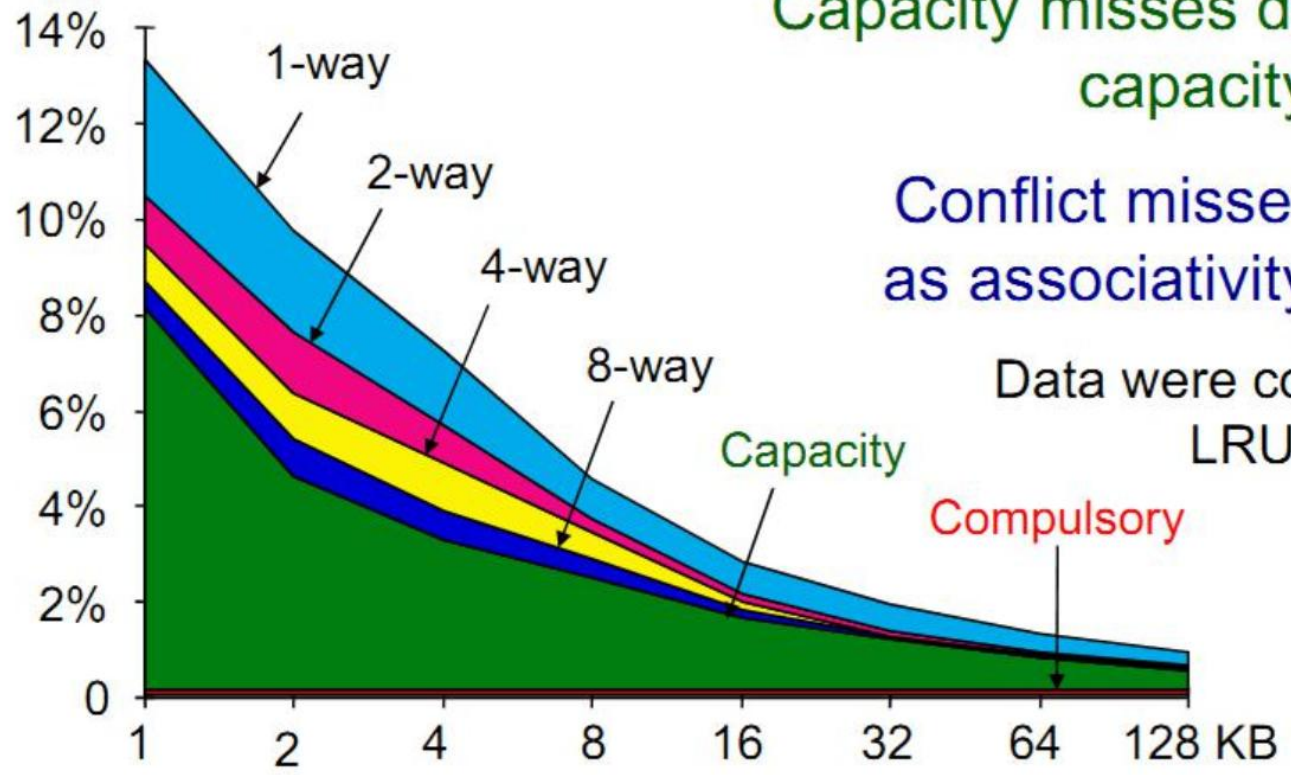
Compulsory misses are independent of cache size

Very small for long-running programs

Capacity misses decrease as capacity increases

Conflict misses decrease as associativity increases

Data were collected using LRU replacement



从统计规律中得到的一些结果

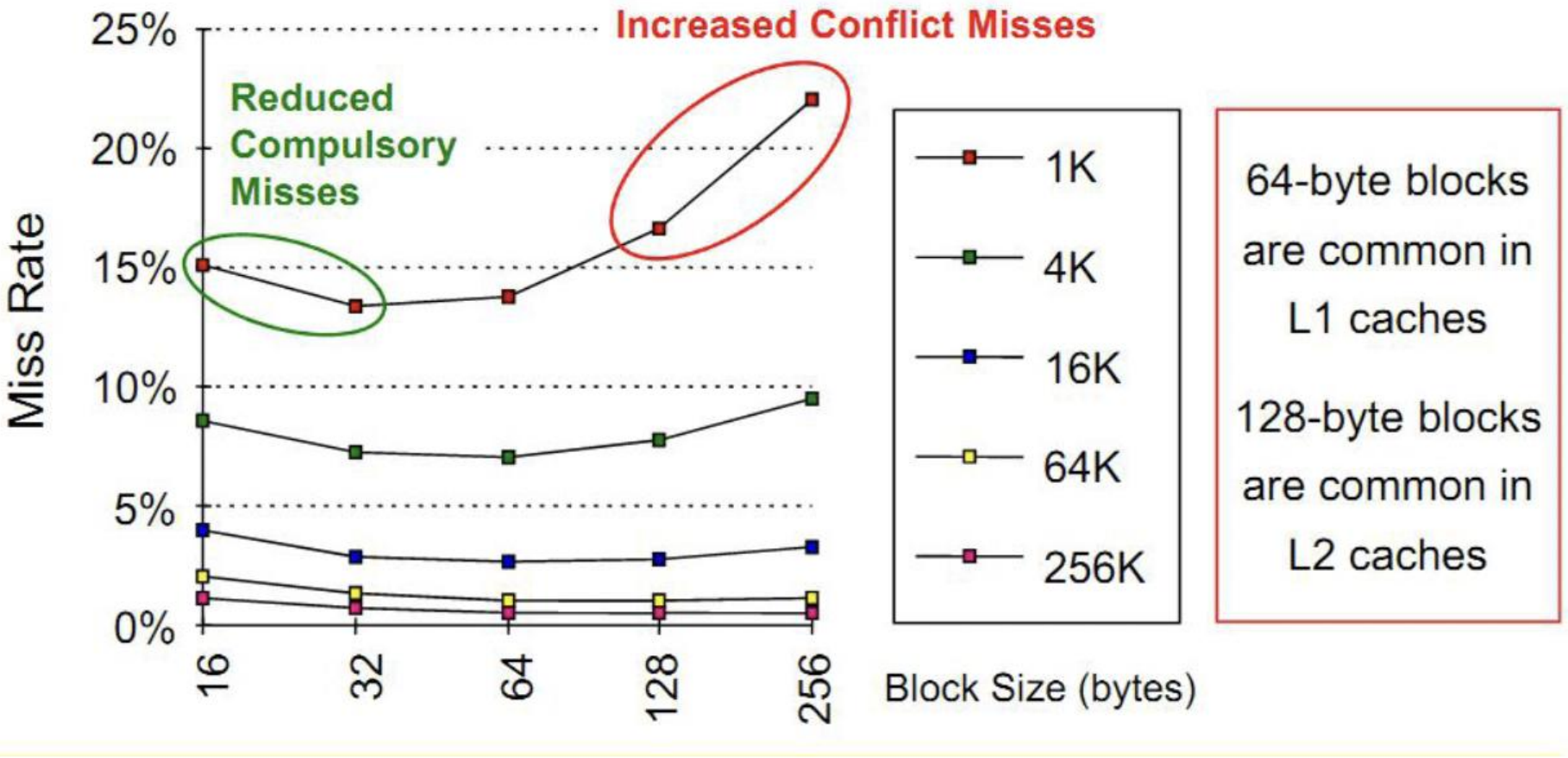
- 相联度越高，冲突失效就越小
- 强制性失效和容量失效不受相联度的影响
- 强制性失效不受Cache容量的影响
- 容量失效随着容量的增加而减少
- 符合2:1Cache经验规则
  - 即大小为N的直接映象Cache的失效率约等于大小为N/2的两路组相联的Cache失效率。

减少3C的方法

从统计规律可知

- 增大Cache容量
  - 对冲突和容量失效的减少有利
- 增大块
  - 减缓强制性失效
  - 可能会增加冲突失效(因为在容量不变的情况下， 块的数目减少了)
- 通过预取可帮助减少强制性失效
  - 必须小心不要把你需要的东西换出去
  - 需要预测比较准确(对数据较困难，对指令相对容易)

减少3C的方法 – 增大块


Increased Conflict Misses
Reduced Compulsory Misses
Miss Rate
25%
20%
15%
10%
5%
0%
16
32
64
128
256
Block Size (bytes)
1K
4K
16K
64K
256K
64-byte blocks are common in L1 caches
128-byte blocks are common in L2 caches

块大小、容量对失效率的影响

- 已知: 不同Cache容量以及块大小条件下的失效率

Block Size	Cache = 4 KB	Cache = 16 KB	Cache = 64 KB	Cache = 256 KB
16 bytes	8.57%	3.94%	2.04%	1.09%
32 bytes	7.24%	2.87%	1.35%	0.70%
64 bytes	7.00%	2.64%	1.06%	0.51%
128 bytes	7.78%	2.77%	1.02%	0.49%
256 bytes	9.51%	3.92%	1.15%	0.49%

- Memory latency = 80 cycles + 1 cycle per 8 bytes
  - Latency of 16-byte block = 80 + 2 = 82 clock cycles
  - Latency of 32-byte block = 80 + 4 = 84 clock cycles
  - Latency of 256-byte block = 80 + 32 = 112 clock cycles
- Which block has smallest AMAT for each cache size?

块大小、容量对AMAT的影响

- Solution: assume hit time = 1 clock cycle
  - Regardless of block size and cache size
- Cache Size = 4 KB, Block Size = 16 bytes
  - AMAT = 1 + 8.57% × 82 = 8.027 clock cycles
- Cache Size = 256 KB, Block Size = 256 bytes
  - AMAT = 1 + 0.49% × 112 = 1.549 clock cycles

Block Size	Cache = 4 KB	Cache = 16 KB	Cache = 64 KB	Cache = 256 KB
16 bytes	AMAT = 8.027	AMAT = 4.231	AMAT = 2.673	AMAT = 1.894
32 bytes	AMAT = **7.082**	AMAT = 3.411	AMAT = 2.134	AMAT = 1.588
64 bytes	AMAT = 7.160	AMAT = **3.323**	AMAT = **1.933**	AMAT = **1.449**
128 bytes	AMAT = 8.469	AMAT = 3.659	AMAT = 1.979	AMAT = 1.470
256 bytes	AMAT = 11.65	AMAT = 4.685	AMAT = 2.288	AMAT = 1.549

块大小、容量的权衡

- 从统计数据可得到如下结论
  - 对于给定Cache容量，块大小增加时，失效率开始是下降，但后来反而上升
  - Cache容量越大，使失效率达到最低的块大小就越大
- 分析
  - 块大小增加，可使强制性失效减少（空间局部性原理）
  - 块大小增加，可使冲突失效增加（Cache中块数量减少）
  - 失效开销增大（上下层间移动，数据传输时间变大）
- 设计块大小的原则，不能只看失效率
  - 原因：平均访存时间 = 命中时间 + 失效率×失效开销

减少3C的方法 - 提高相联度

- 8路组相联在降低失效率方面的作用已经和全相联一样有效
- 2:1Cache经验规则
  - 容量为N的直接映象Cache失效率与容量为N/2的两路组相联Cache的失效率差不多相同
- 提高相联度，会增加命中时间

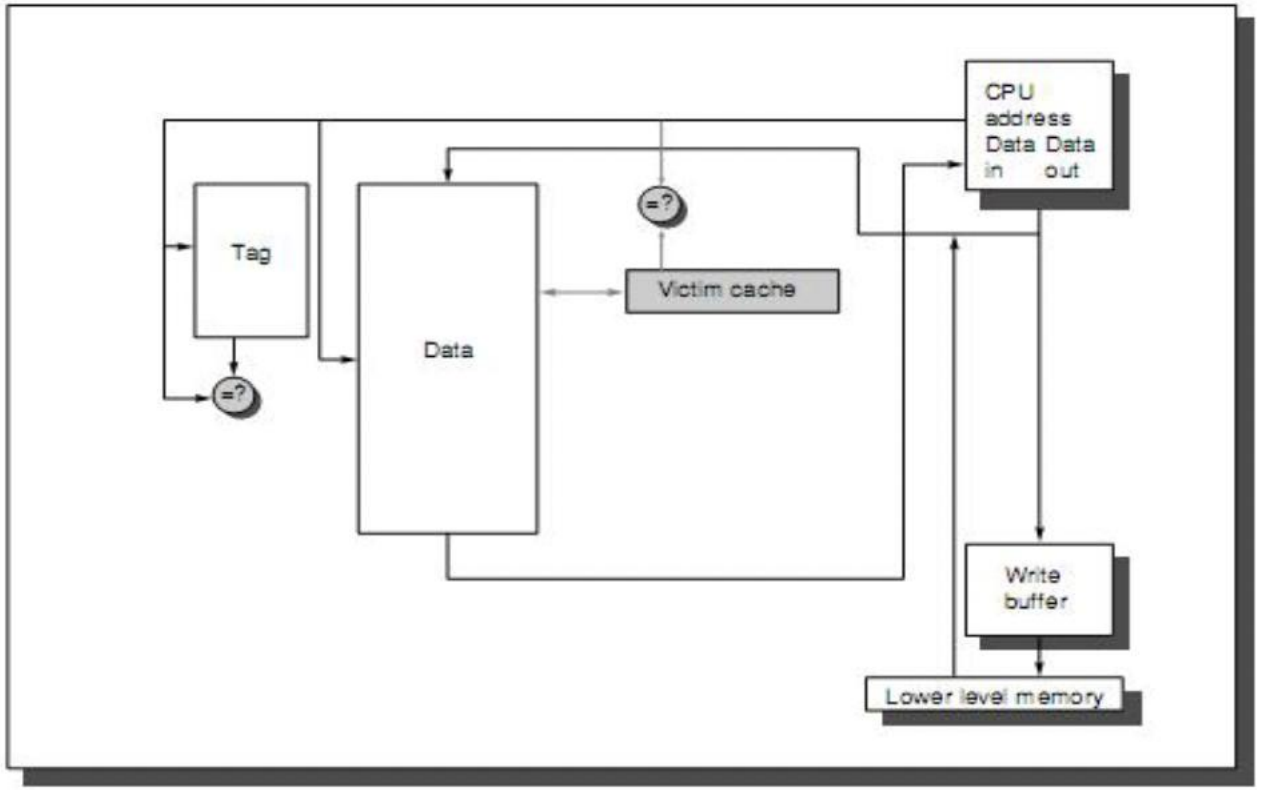
Size (KB)	1-way	2-way	4-way	8-way
1	7.65	6.60	6.22	5.44
2	5.90	4.90	4.62	4.09
4	4.60	3.95	3.57	3.19
8	3.30	3.00	2.87	2.59
16	2.45	2.20	2.12	2.04
32	2.00	1.80	1.77	1.79
64	1.70	1.60	1.57	1.59
128	1.50	1.45	1.42	1.44

Average Memory Access Time

OOPS!

Victim Cache(1/2)

- 在Cache和Memory之间增加一个小的全相联Cache



Victim Cache(2/2)

- 基本思想
  - 通常Cache为直接映象时冲突失效率较大
  - Victim cache采用全相联 - 失效率较低
  - Victim cache存放由于（冲突）失效而被丢弃的那些块
  - 失效时，首先检查Victim cache是否有该块，如果有就将该块与Cache中相应块互换
- Jouppi (DEC SRC)发现
  - 含1到5项的Victim cache对减少失效很有效，尤其是对于那些小型的直接映象数据Cache
  - 测试结果，项为4的Victim Cache能使4KB直接映象数据Cache冲突失效减少20%-90%

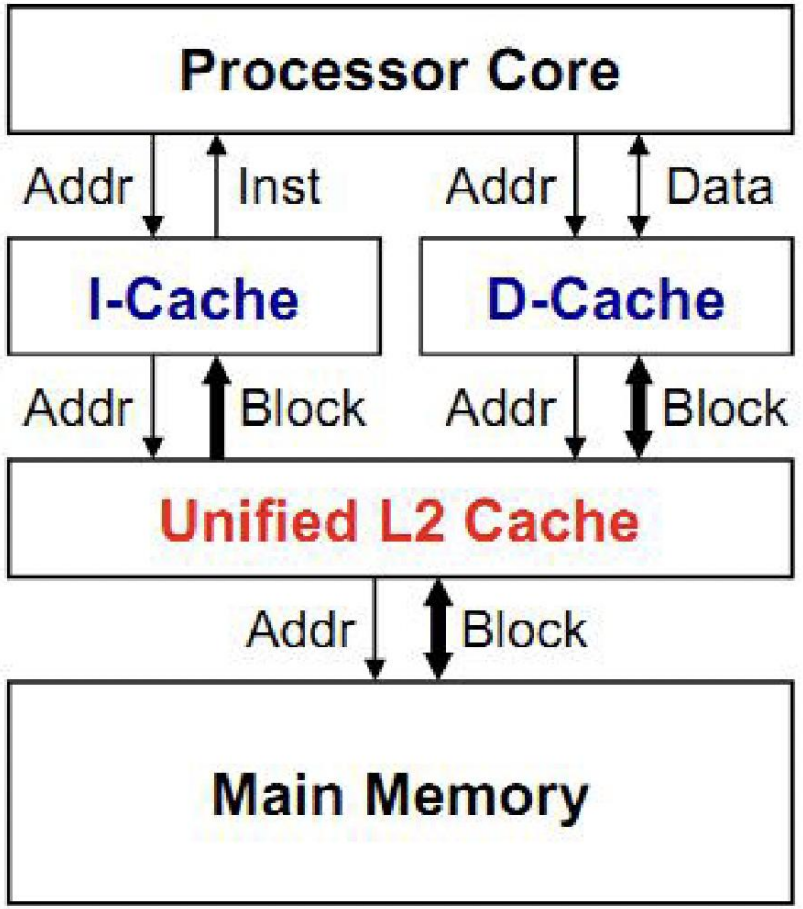
减少失效开销

平均访存时间=命中时间+失效率×失效开销

- 减少CPU与存储器间性能差异的重要手段
- 基本手段：
  - 1、多级Cache技术(Multilevel Caches)
  - 2、让读优先于写(Giving Priority to Read Misses over Writes)

采用多级Cache

- 一级cache保持较小容量
  - 降低命中时间
  - 降低每次访问的能耗
- 增加二级cache
  - 减少与存储器的gap
  - 减少存储器总线的负载
- 多级cache的优点
  - 减少失效开销
  - 缩短平均访存时间（AMAT）
- 较大容量的L2 cache可以捕捉许多L1 cache的失效
  - 降低全局失效率



多级包容性（Multilevel Inclusive）

- L1 cache 的块总是存在于L2 cache中
  - 浪费了L2 cache 空间，L2 还应当有存放其他块的空间
- L1中miss, 但在L2中命中，则从L2拷贝相应的块到L1
- 在L1和L2中均miss, 则从更低级拷贝相应的块到L1和L2
- 对L1写操作导致将数据同时写到L1和L2
- Write-through 策略用于L1到L2
- Write-back 策略可用于L2 到更低级存储器，以降低存储总线的数据传输压力
- L2的替换动作（或无效动作）对L1可见
  - 即L2的一块被替换出去，那么其在L1中对应的块也要被替换出去
- L1和L2的块大小可以相同也可以不同
  - Pentium 4 had 64-byte blocks in L1 but 128-byte blocks in L2
  - Core i7 uses 64-byte blocks at all cache levels (simpler)

多级不包容（Multilevel Exclusive）

- L1 cache 中的块不会在L2 cache中，以避免浪费空间
- 在L1中miss, 但在L2中命中，将导致Cache间块的互换
- 在L1和L2均miss, 将仅仅从更低层拷贝相应的块到L1
- L1的被替换的块移至L2
  - L2 存储L1抛弃的块，以防后续L1还需要使用
- L1到L2的写策略为 Write-Back
- L2到更低级cache的写策略为 Write-Back
- AMD Athlon： 64KB L1 、 256KB L2

多级cache的性能分析

- **局部失效率**：该级Cache的失效次数 / 到达该级Cache的访存次数
  - Miss rateL1 for L1 cache
  - Miss rateL2 for L2 cache
- **全局失效率**：该级Cache的失效次数/ CPU发出的访存总次数
  - Miss rateL1 for L1 cache
  - Miss rateL1 × Miss rateL2 for L2 cache
  - 全局失效率是度量L2 cache性能的更好方法
- **性能参数**
  - AMAT = Hit TimeL1+Miss rateL1×Miss penaltyL1
  - Miss penaltyL1 = HitTimeL2 + Miss rateL2×Miss penaltyL2
  - AMAT = Hit TimeL1+Miss rateL1× (Hit TimeL2+Miss rateL2×Miss penaltyL2)

L1 cache 失效率

- 对于I-Cache和D-Cache分开的L1 Cache

  Miss RateL1 = %inst × Miss RateI-Cache + %data × Miss RateD-Cache

  %inst = Percent of Instruction Accesses = 1 / (1 + %LS)

  %data = Percent of Data Accesses = %LS / (1 + %LS)

  %LS = Frequency of Load and Store instructions

- 每条指令的L1 失效次数:

  Misses per InstructionL1 = Miss RateL1 × (1 + %LS)

  Misses per InstructionL1 = Miss Rate_I-Cache + %LS × Miss Rate_DCache

具有二级Cache的AMAT举例

- Problem: 计算AMAT
  - I-Cache 失效率 = 1%， D-Cache失效率 = 10%
  - L2 Cache失效率 = 40%
  - L1 命中时间 = 1 cycle (I-Cache 和D-Cache相同)
  - L2 命中时间 = 8 cycles， L2 失效开销 = 100 cycles
  - Load + Store 指令频度 = 25%
- Solution:
  - 平均每条指令访存次数 = 1+25% = 1.25
  - 平均每条指令的失效次数 = 1%+25%× 10% = 0.035
  - L1的失效率= 0.035/1.25=0.028
  - L1的失效开销 = 8 + 0.4×100 = 48 cycles
  - AMAT = 1+0.028×48 = 2.344

Memory Stall Cycles Per Instruction

Memory Stall Cycles per Instruction

= Memory Access per Instruction × Miss RateL1 × Miss PenaltyL1

= (1 + %LS) × Miss RateL1 × Miss PenaltyL1

= (1 + %LS) × Miss RateL1 × (Hit TimeL2 + Miss RateL2 × Miss PenaltyL2)

Memory Stall Cycles per Instruction

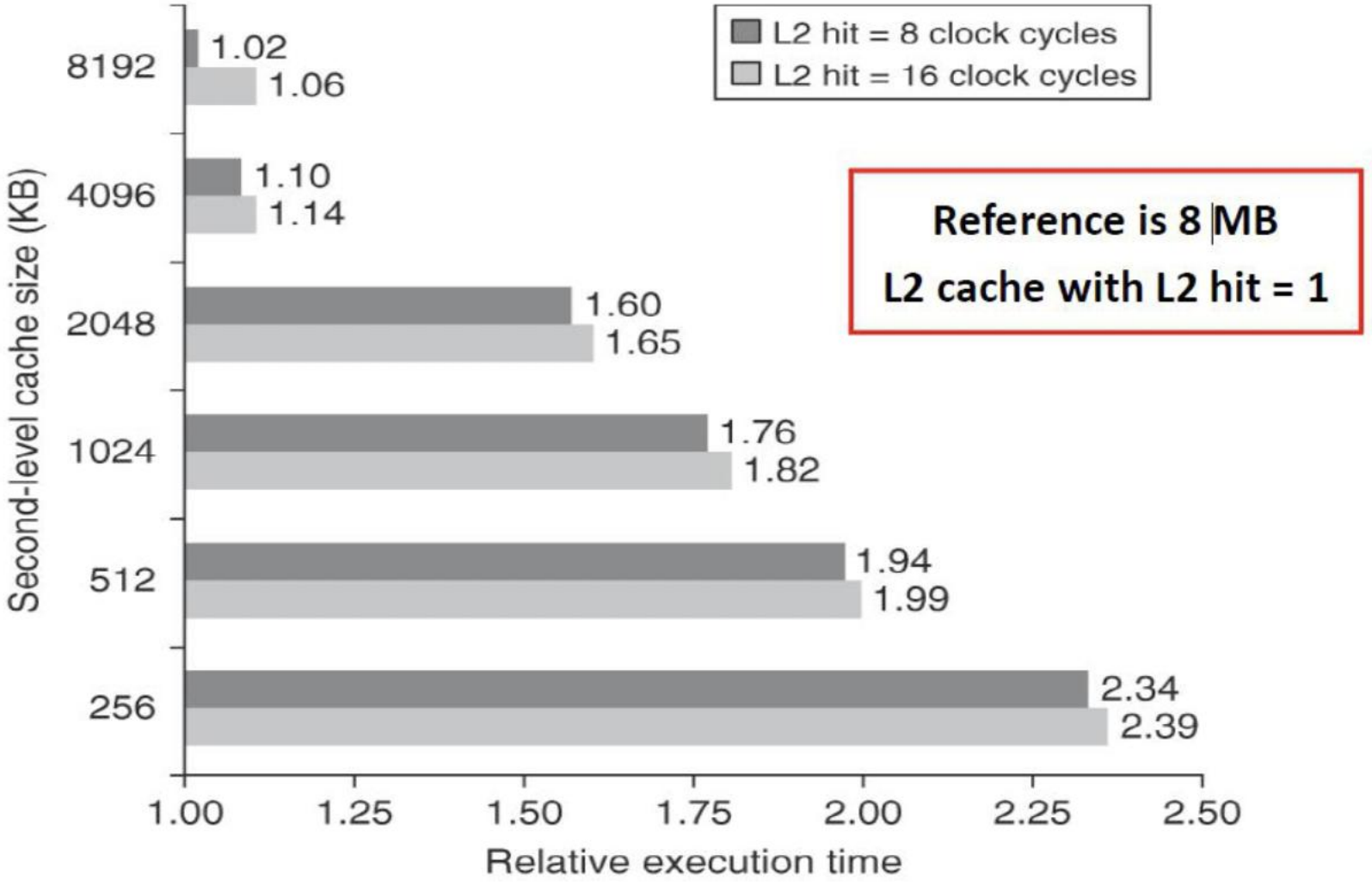
= Misses per InstructionL1 × Hit TimeL2 + Misses per InstructionL2 × Miss PenaltyL2

- Misses per InstructionL1 = (1 + %LS) × Miss RateL1
- Misses per InstructionL2 = (1 + %LS) × Miss RateL1 × Miss RateL2

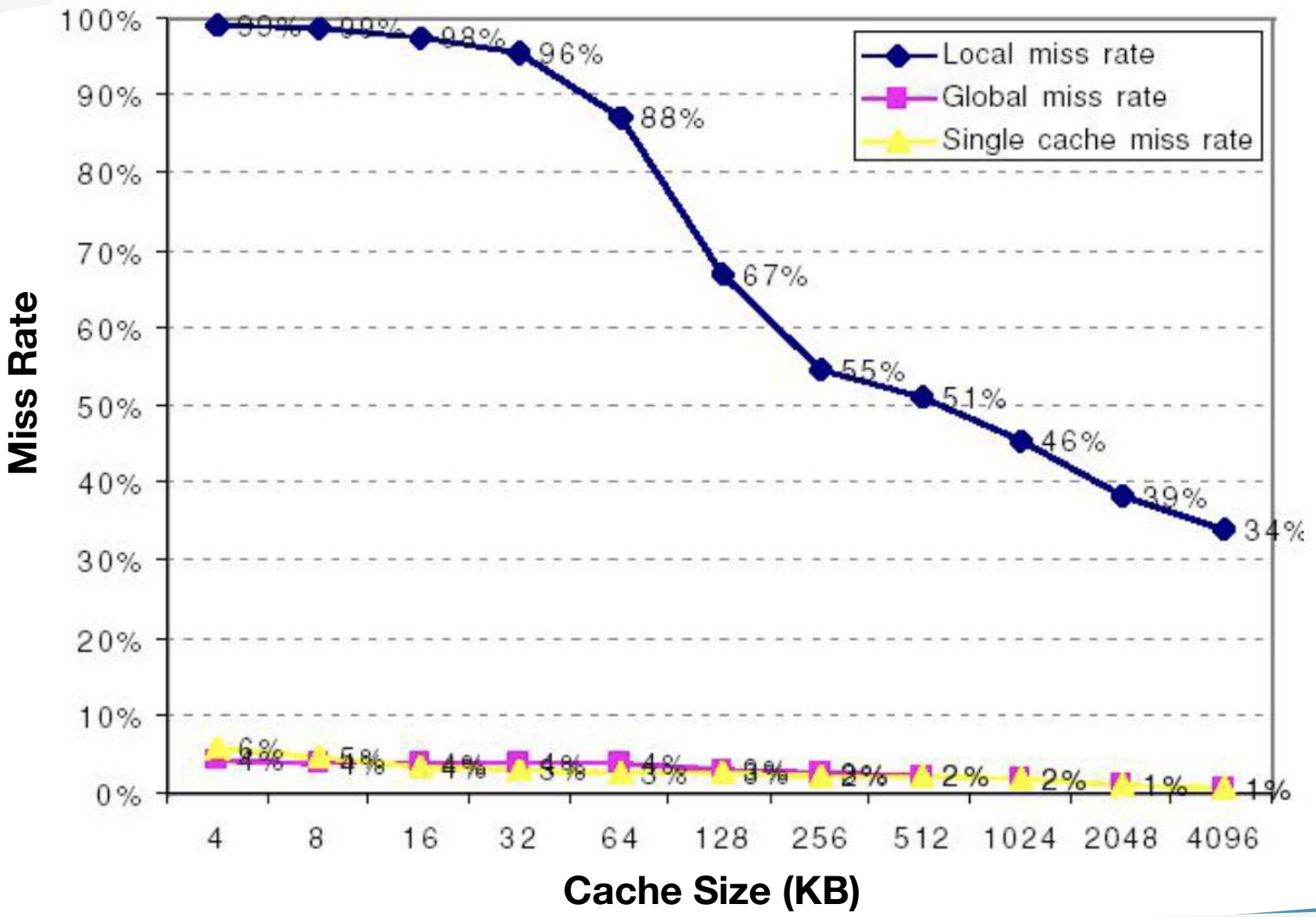
两级Cache的性能

- Problem: 程序运行产生1000个存储器访问
  - – I-Cache misses = 5, D-Cache misses = 35, L2 Cache misses = 8
  - – L1 Hit = 1 cycle, L2 Hit = 8 cycles, L2 Miss penalty = 80 cycles
  - – Load + Store frequency = 25%, $CPI_{execution}$ = 1.1 (perfect cache)
  - – 计算memory stall cycles per instruction 和有效的CPI
  - – 如果没有L2 cache, 有效的CPI是多少？
- Solution:
  - – L1 Miss Rate = (5 + 35) / 1000 = 0.04 (or 4% per access)
  - – L1 misses per Instruction =0.04 × (1 + 0.25) = 0.05
  - – L2 misses per Instruction =(8 / 1000) × 1.25 = 0.01
  - – Memory stall cycles per Instruction =0.05 × 8 + 0.01 × 80 = 1.2
  - – CPI_{L1+L2} = 1.1 + 1.2 = 2.3, $CPI/CPI_{execution}$ = 2.3/1.1 = 2.1x slower
  - – $CPI_{L1-only}$ =1.1 + 0.05 × 80 = 5.1 (worse)

不同大小的L2cache对应的执行时间



Miss Rates vs. Cache Size for Multilevel Caches



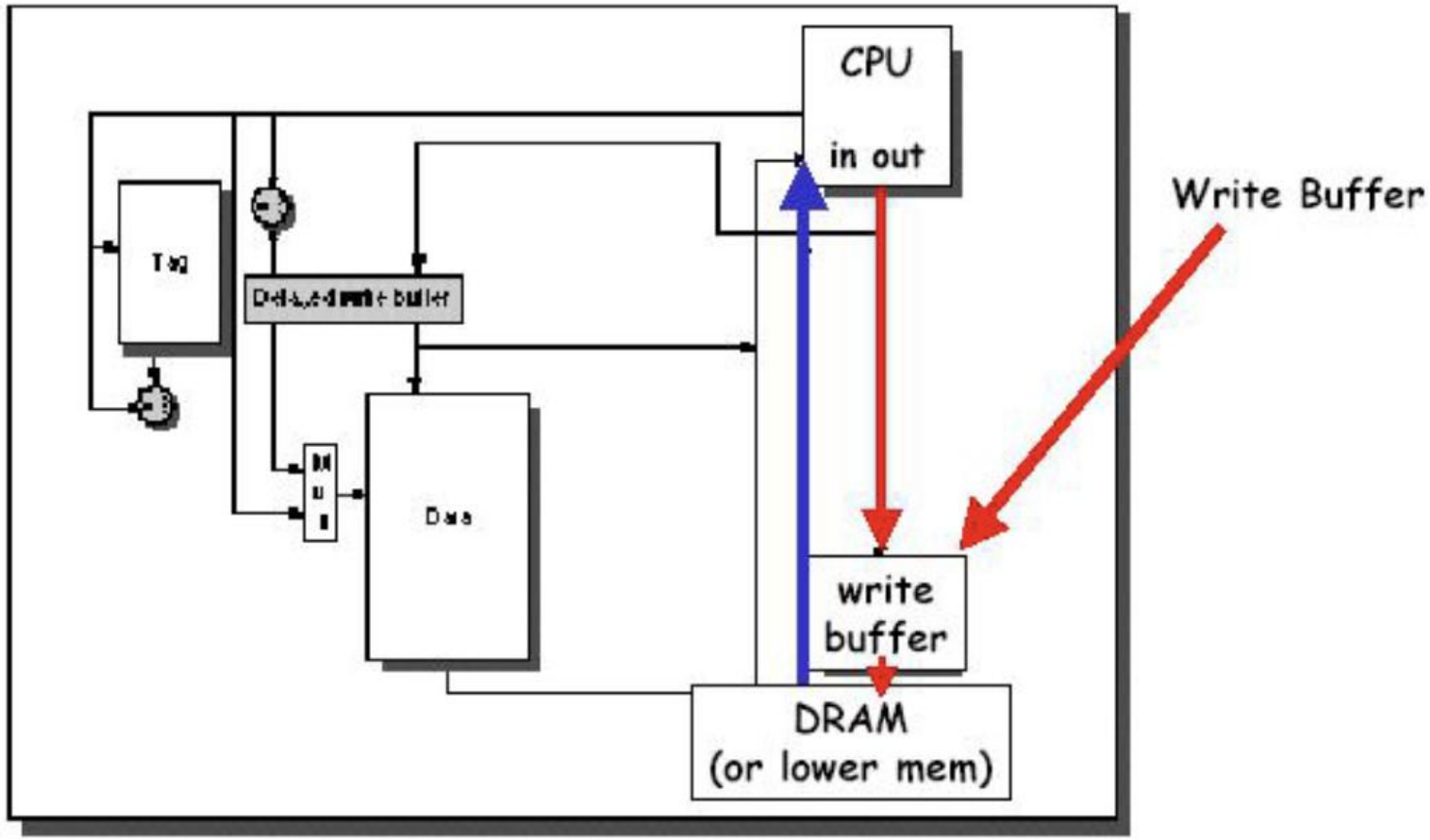
两级Cache的一些研究结论

- 在L2比L1大得多的情况下，两级Cache全局失效率和容量与第二级Cache相同的单级Cache的失效率接近
- 局部失效率不是衡量第二级Cache的好指标
  - 它是第一级Cache失效率的函数
  - 不能全面反映两级Cache体系的性能
- 第二级Cache设计需考虑的问题
  - 容量：一般很大，可能没有容量失效，只有强制性失效和冲突失效
    - 相联度对第二级Cache的作用
    - Cache可以较大，以减少失效次数
  - 多级包容性问题：第一级Cache中的数据是否总是同时存在于第二级Cache中
- 如果L1和L2的块大小不同，增加了多级包容性实现的复杂性

多级Cache举例

- Given the data below, what is the impact of second-level cache associativity on its miss penalty?
  - Hit timeL2 for direct mapped = 10 clock cycles
  - Two-way set associativity increases hit time by 0.1 clock cycles to 10.1clock cycles
  - Local miss rateL2 for direct mapped = 25%
  - Local miss rateL2 for two-way set associative = 20%
  - Miss penaltyL2 = 50 clock cycles
- 结论：提高相联度，可减少第一级Cache的失效开销
- 第二级Cache特点：容量大，高相联度，块较大，重点减少失效次数

让读优先于写图示



让读失效优先于写（1/2）

- 由于读操作为大概率事件，需要读失效优先，以提高性能
- Write-Through Cache ->Write Buffer (写缓冲)，特别对写直达法更有效
  - Write Buffer: CPU不必等待写操作完成，即将要写的数据和地址送到Write Buffer后，CPU继续作其他操作
  - 写缓冲导致对存储器访问的复杂化
    - 在读失效时写缓冲中可能保存有所读单元的最新值，还没有写回
    - 例如，直接映射、写直达、512和1024映射到同一块。则
    - SW R3, 512(R0)
    - LW R1, 1024(R0) 失效
    - LW R2, 512(R0) 失效
  - – 解决问题的方法
    - • 推迟对读失效的处理，直到写缓冲器清空，导致新的问题——读失效开销增大。
    - • 在读失效时，检查写缓冲的内容，如果没有冲突，而且存储器可访问，就可以继续处理读失效

让读失效优先于写（2/2）

- Write-Back Cache ->Victim Buffer
  - – 写回法时，也可以利用写缓冲器来提高性能
    - 把脏块放入缓冲区，然后从存储器调读操作需要的块，最后再把缓冲器中的内容写入存储器。
  - – 被替换的脏块放到了victim buffer
  - – 在脏块被写回前，需要处理读失效
  - – 问题: victim buffer 可能含有该读失效要读取的块
    - Solution: 查找victim buffer，如果命中直接将该块调入Cache

缩短命中时间

平均访存时间=命中时间+失效率×失效开销

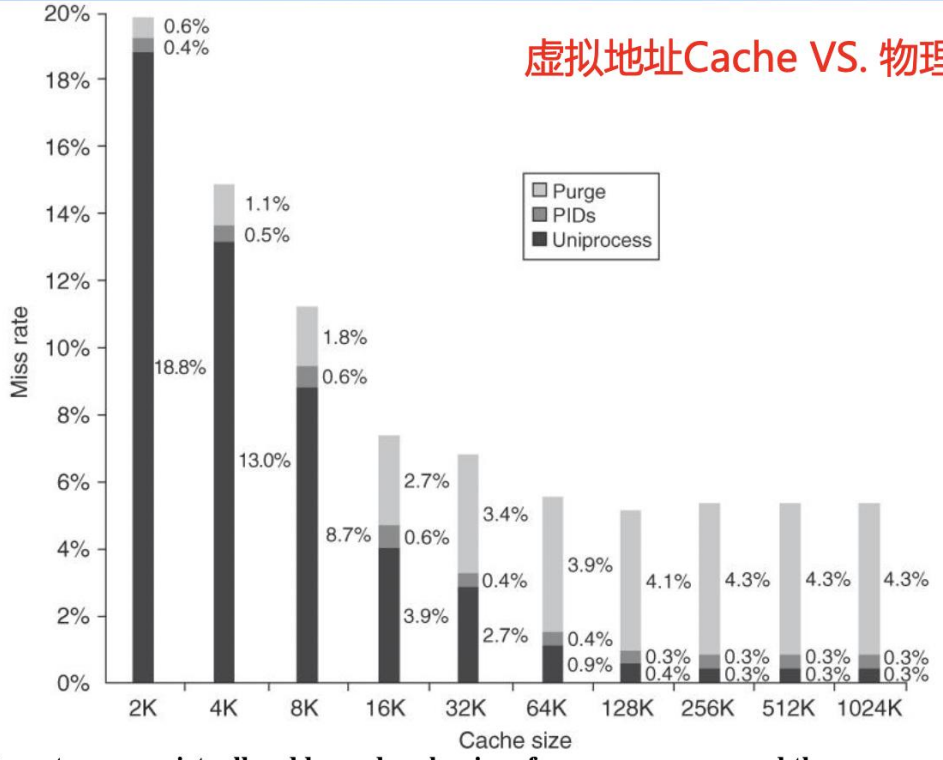
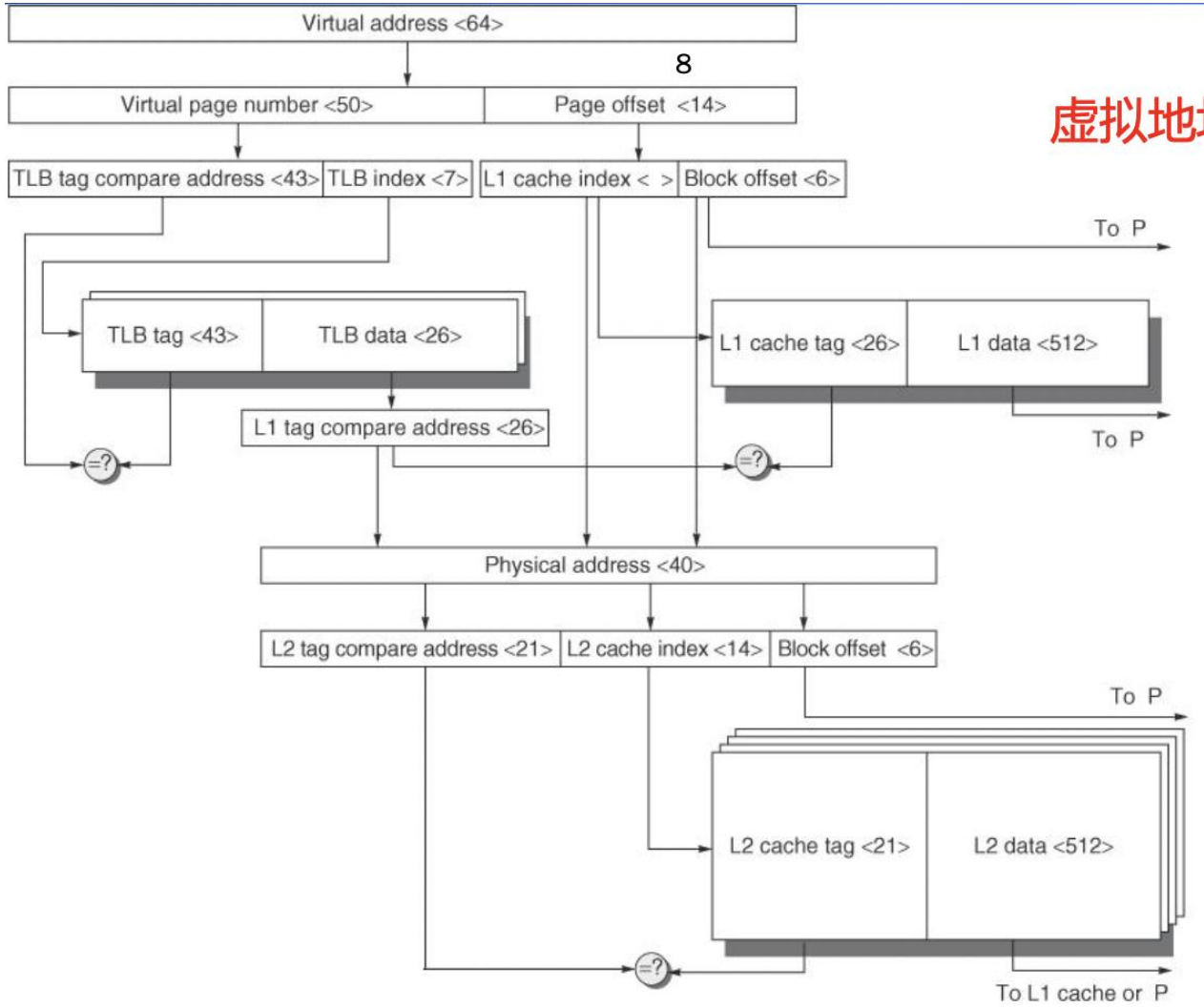


Figure B.16 Miss rate versus virtually addressed cache size of a program measured three ways: without process switches (uniprocess), with process switches using a process-identifier tag (PID), and with process switches but without PIDs (purge). PIDs increase the uniprocess absolute miss rate by 0.3% to 0.6% and save 0.6% to 4.3% over purging. Agarwal [1987] collected these statistics for the Ultrix operating system running on a VAX, assuming direct-mapped caches with a block size of 16 bytes. Note that the miss rate goes up from 128K to 256K. Such nonintuitive behavior can occur in caches because changing size changes the mapping of memory blocks onto cache blocks, which can change the conflict miss rate.

缩短命中时间

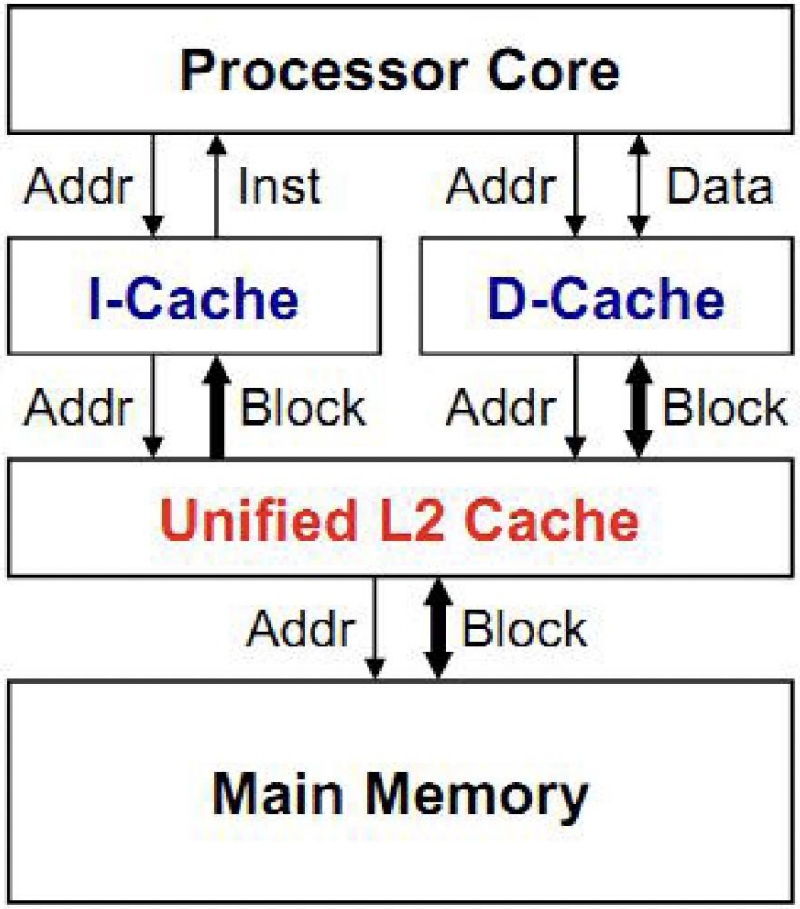


虚拟地址转换与Cache定位 并行

小结

基本Cache优化方法

- 降低失效率: 引起失效的3C
  - 1、增加Cache块的大小
  - 2、增大Cache容量
  - 3、提高相联度
- 减少失效开销
  - 4、多级Cache
  - 5、使读失效优先于写失效
- 缩短命中时间
  - 6、避免在索引缓存期间进行地址转换





授课教师：王强
邮箱：qiang.wang@hit.edu.cn

谢谢

Thank You